

Matt Pocock 스킬 튜토리얼 — 공지사항 게시판을 배포까지

먼저 읽는 용어 정리

AI와 대화

- **에이전트** == 요청을 받아 자료를 읽고 도구로 작업하는 AI
- **코딩 에이전트** == 이 가이드에서는 Claude Code·Codex·안티그래비티처럼 AI와 함께 코드를 읽고 수정하고 실행하는 프로그램을 뜻함
- **코딩 에이전트 프로그램** == 코딩 에이전트. 이 가이드에서는 같은 뜻으로 사용
- **메인 에이전트(주 에이전트)** == 코딩 에이전트 프로그램 안에서 특정 작업의 사용자 요청을 중심에서 맡고 전체 결과를 정리하는 AI. 여러 작업을 열면 각 작업에 메인 에이전트가 있을 수 있음
- **서브 에이전트** == 메인 에이전트가 나눠 맡긴 일을 처리하는 AI
- **세션** == 하나의 에이전트와 이어지는 대화 흐름. 메시지 한 번이 아니라 여러 질문·답변·작업이 이어지는 구간
- **스킬** == 에이전트가 특정 작업을 할 때 따르는 안내서
- **하네스** == 에이전트가 파일을 읽고 도구를 실행할 수 있게 해 주는 환경
- **컨텍스트** == 에이전트가 지금 참고하는 대화·지침·읽어 온 자료
- **토큰** == AI가 글과 코드를 처리할 때 나누는 작은 조각. 글자 수와 정확히 같지는 않음
- **컨텍스트 용량** == 모델이 한 번에 다룰 수 있는 내용의 한도

계획과 작업 관리

- **소프트웨어 개발** == 추상적 의도 + 질문을 통한 구체적 결정(기획/기술) + 구현·검증·피드백의 반복
 1. **예약 서비스** == “예약을 편하게 하고 싶다” + 예약 시간·취소 규칙·중복 예약 방지 방식 결정 + 구현하고 실제 예약 과정에서 검증·개선
 2. **쇼핑몰** == “온라인으로 상품을 팔고 싶다” + 상품 구성·결제 수단·재고 관리 방식 결정 + 구현하고 구매 과정에서 검증·개선
 3. **게임** == “친구들과 즐길 게임을 만들고 싶다” + 게임 규칙·참여 인원·실시간 연결 방식 결정 + 구현하고 함께 플레이하며 검증·개선
- **기획 질문 → 기획 결정** == 무엇을 누구를 위해 만들지 묻고 정하는 것. “누가 공지를 작성하나요?” → “학교 공지 담당자 한 명만 작성한다”
- **기술 질문 → 기술 결정** == 정한 기능을 어떤 방법으로 구현할지 묻고 정하는 것. “로그인 상태를 어떻게 유지하나요?” → “30분 토큰을 프론트 메모리에 보관한다”
- **워크플로우** == 상황에 맞게 작업을 이어 가는 순서
- **기획문서** == 누구를 위해 무엇을 만들지 설명한 자료
- **명세** == 어떤 조건에서 어떻게 동작해야 하는지 합의한 구현 기준
- **구현 티켓** == 합의한 기능을 만들고 검사해야 끝나는 작업 카드
- **이슈 트래커** == 질문·작업·담당자·진행 상태를 모아 관리하는 곳

- **CONTEXT.md** == 우리 프로젝트에서 쓰는 업무 용어의 뜻을 적은 파일
- **ADR** == 중요한 설계 선택을 왜 했는지 남긴 기록

조사와 개발

- **웨이파인더(wayfinder)** == 그릴위드독스보다 더 알아보고 실험해야 하는 큰 기획에서, 조사·시제품·질문과 답변을 연결해 설계를 구체화하는 스킬
- **트리아지(triage)** == 들어온 요청을 분류하고, 부족한 정보를 확인해 다음 처리를 정하는 일
- **그릴링(grilling)** == 애매한 계획을 질문으로 구체화하고 합의하는 일
- **리서치(research)** == 모르는 사실을 자료와 출처로 확인하는 일
- **프로토타입(prototype)** == 설계 질문에 답하려고 만드는 시험용 모형
- **디버깅(debugging)** == 고장을 재현하고 원인을 찾아 고치는 일
- **재현** == 같은 조건에서 같은 문제가 다시 발생하는지 확인하는 일
- **로그** == 프로그램이 실행하면서 남긴 기록
- **테스트** == 실제 결과가 기대한 결과와 같은지 검사하는 일
- **회귀 테스트** == 변경 후 이전 기능이나 고친 문제가 다시 깨지지 않았는지 확인하는 검사
- **리팩터링(refactoring)** == 사용자가 보는 동작을 유지하면서 코드 내부를 정리하는 일

코드 기록과 배포

- **저장소(repository)** == 프로젝트 파일과 변경 이력을 관리하는 곳
- **Git** == 코드의 변경 이력을 관리하는 도구
- **GitHub** == 원격 Git 저장소와 협업 기능을 제공하는 서비스
- **브랜치(branch)** == 같은 프로젝트에서 작업을 따로 이어 가는 이력의 갈래
- **워크트리(worktree)** == 같은 Git 저장소의 작업을 별도로 펼쳐 놓는 폴더
- **커밋(commit)** == 선택한 변경을 Git 이력에 기록하는 일
- **푸시(push)** == 내 커밋을 원격 저장소로 보내는 일
- **PR** == 브랜치의 변경을 검토하고 합쳐 달라는 요청
- **머지(merge)** == 다른 갈래의 변경을 합치는 일
- **충돌** == 서로 다른 변경을 어떻게 합칠지 판단이 필요한 상태
- **빌드(build)** == 코드를 실행·배포할 수 있는 형태로 준비하는 일
- **배포(deploy)** == 새 결과물을 실제 실행 환경에 반영하는 일
- **CI/CD** == 변경 검사와 배포 준비·실행을 자동화하는 흐름. 배포 전 승인 여부는 설정에 따라 다름

헛갈리는 용어 비교

비교	차이
코딩 에이전트 vs 메인 에이전트	코딩 에이전트 = 작업에 사용하는 프로그램 전체 / 메인 에이전트 = 그 안에서 특정 작업을 맡는 AI. 프로그램과 작업 담당자의 차이
세션 vs 메인 에이전트	세션 = 이어지는 대화 흐름 / 메인 에이전트 = 그 작업을 중심에서 맡는 AI
메인 vs 서브 에이전트	메인 = 전체 요청 담당 / 서브 = 나눠 맡은 부분 담당
에이전트 vs 스킬	에이전트 = 일하는 주체 / 스킬 = 그 주체가 따르는 안내서
스킬 vs 워크플로우	스킬 = 특정 작업의 안내 / 워크플로우 = 작업들을 이어 가는 순서
세션 vs 컨텍스트	세션 = 대화가 이어지는 구간 / 컨텍스트 = 지금 참고하는 내용
컨텍스트 vs CONTEXT.md	컨텍스트 = 지금 참고하는 자료 전체 / CONTEXT.md = 업무 용어를 적은 특정 파일
컨텍스트 용량 vs 토큰	용량 = 한 번에 처리할 수 있는 한도 / 토큰 = 처리량을 세는 조각 단위
트리아지 vs 그릴링	트리아지 = 접수된 요청의 상태와 다음 처리 정리 / 그릴링 = 계획의 모호한 조건을 질문으로 합의
리서치 vs 그릴링	리서치 = 사실 확인 / 그릴링 = 선택할 내용 합의
트리아지 vs 디버깅	트리아지 = 요청을 검토해 다음 처리 정하기 / 디버깅 = 실제 고장의 원인 찾아 수정하기
grill-with-docs vs wayfinder	grill-with-docs = 질문하며 설계와 기록을 정리 / wayfinder = 더 알아보고 실험해야 하는 큰 기획에서 조사·시제품·논의를 연결
기획문서 vs 명세	기획문서 = 목적과 원하는 기능 / 명세 = 구체적으로 합의한 동작·검증 기준. 이름보다 실제 내용이 중요
프로토타입 vs 구현	프로토타입 = 판단을 위한 실험 / 구현 = 실제 사용할 기능 만들기
브랜치 vs 워크트리	브랜치 = 작업 이력의 갈래 / 워크트리 = 파일을 펼쳐 놓는 작업 폴더
저장 vs 커밋 vs 푸시	저장 = 파일에 반영 / 커밋 = Git 이력에 기록 / 푸시 = 원격으로 전송
PR vs 머지	PR = 검토·합치기 요청 / 머지 = 실제로 변경 합치기
빌드 vs 배포	빌드 = 실행할 결과물 준비 / 배포 = 실행 환경에 반영

기억할 것: 새 세션에 이전 내용이 모두 자동 전달되는 것은 아닙니다. 서브 에이전트의 작업 완료는 전체 작업 완료와 다르고, 푸시·빌드 성공도 실제 서비스의 정상 동작을 보장하지 않습니다.

스킬은 어떤 기능을 묶어놓았나요?

각 행은 **스킬** = **기능** 구성으로 읽으면 됩니다. + 는 함께 묶인 기능을 뜻합니다. 실제 내부 호출 순서를 뜻하지는 않습니다. 예를 들어 `grill-with-docs` 는 `grill-me` 를 직접 호출하지 않고, 같은 `grilling` 에 `domain-modeling` 을 더해 사용합니다.

`research` 와 `prototype` 은 각각 단독으로도 사용할 수 있습니다. 사실 확인만 필요하면 `research` , 만들어서 확인할 질문이 뚜렷하면 `prototype` , 조사·실험·추가 논의를 연결해야 하면 `wayfinder` 를 사용합니다.

스킬	기능 구성	쉽게 말하면
<code>grilling</code>	질문 + 사용자 답변 + 모호한 조건 확인	애매한 생각을 질문으로 구체화하기
<code>grill-me</code>	<code>grilling</code>	질문하며 생각을 구체화하기
<code>domain-modeling</code>	용어 정리 + 중요한 설계 결정 기록(ADR)	프로젝트의 말뜻과 선택의 이유 남기기
<code>grill-with-docs</code>	<code>grill-me</code> + <code>domain-modeling</code>	질문하며 구체화하기 + 용어·결정 기록
<code>research</code>	서브 에이전트 조사 + 공식 문서·원본 코드 등 1차 자료 확인 + 출처가 있는 조사 기록	모르는 사실을 찾아 근거와 함께 정리하기
<code>prototype</code>	설계 질문 + 시험용 화면·동작 구현 + 직접 사용한 피드백	만들어봐야 알 수 있는 것을 작게 실험하기
<code>wayfinder</code>	<code>grill-with-docs</code> + 필요할 때 <code>research</code> + 필요할 때 <code>prototype</code> + 결정을 위한 준비 작업 + 결과를 반영한 추가 논의	더 알아보고 실험해야 하는 큰 기획을 구체화하기
<code>to-spec</code>	기존 논의 정리 + 동작·설계 결정·검증 기준 명세화	합의한 내용을 구현 기준으로 문서화하기
<code>to-tickets</code>	명세·계획 분해 + 작업별 완료 조건 + 선행 작업 연결	하나씩 만들고 확인할 구현 작업으로 나누기
<code>codebase-design</code>	모듈 책임·인터페이스 설계 + 구조를 판단하는 공통 기준	코드를 어디서 나누고 어떻게 연결할지 정하기
<code>tdd</code>	실패하는 테스트 + 통과시키는 구현 + 리팩터링	테스트를 먼저 쓰고 작은 단위로 구현하기
<code>implement</code>	구현 + 가능한 곳에서 <code>tdd</code> + 마무리 <code>code-review</code>	만들고 테스트하고 검토하기
<code>code-review</code>	코딩 규칙 검토 + 명세 일치 검토	프로젝트 규칙을 지켰는지와 요청한 대로 만들었는지 확인
<code>triage</code>	요청 분류·상태 관리 + 필요할 때 <code>grill-with-docs</code> 에 해당하는 논의·기록	들어온 요청을 정리하고 부족한 조건을 구체화
<code>improve-codebase-architecture</code>	코드 구조 조사 + <code>codebase-design</code> + 개선 후보 제안 + 선택한 후보에 대한 논의·기록	구조를 살펴보고 무엇을 어떻게 개선할지 정하기

초간단 요약

1. 준비 후 전체 스킬 설치 — 먼저 [시작 전 준비](#)를 마칩니다. macOS는 터미널, Windows는 PowerShell에 아래 명령을 붙여 넣습니다.

```
npx --yes skills@latest add mattpocock/skills --global --skill "*" --agent codex claude-code --copy --yes
```

설치 후 Claude Code·Codex를 완전히 종료하고 다시 실행합니다. 그래도 스킬이 보이지 않으면 설치 상태를 확인하고 PC를 재부팅합니다.

2. 프로젝트 폴더를 열고 아래 순서로 진행 — 대화창에서 하나씩 호출합니다.

순서	스킬	하는 일
프로젝트 최초 1회만	<code>/setup-matt-pocock-skills</code>	이 프로젝트의 작업 규칙 설정. 명세·티켓은 파일 관리 추천
무엇을 만들지 정하기	<code>/grill-with-docs</code>	질문에 답하며 기능을 정하고, 용어와 결정 이유를 기록
만들 내용 정리	<code>/to-spec</code>	함께 정한 기능과 완성 조건을 문서로 작성
작은 일로 나누기	<code>/to-tickets</code>	하나씩 만들고 확인할 작업으로 나누기
실제로 만들기	<code>/implement</code>	명세·티켓을 읽고 코드 작성과 테스트 진행

`/setup-matt-pocock-skills` 는 프로젝트를 처음 시작할 때 단 한 번만 호출합니다. 같은 프로젝트에서 새 기능을 추가하거나, 새 대화를 열거나, 에이전트를 재시작해도 다시 호출하지 않습니다.

각 단계의 결과를 확인한 뒤 다음으로 넘어갑니다. 초기 설정이 끝난 프로젝트의 새 기능은 보통 `/grill-with-docs` 부터 시작합니다.

같은 대화에서 앞 단계가 끝났다면 다음 스킬 이름만 입력하면 됩니다. `/grill-with-docs` 에서 합의한 뒤에는 `/to-spec` , 명세를 확인한 뒤에는 `/to-tickets` , 작업 범위를 정한 뒤에는 `/implement` 로 이어갑니다. 앞서 설명한 요구사항·문서 경로·테스트 조건을 매번 다시 붙여 넣지 않습니다. 에이전트가 기존 대화와 프로젝트 문서를 바탕으로 이어가고, 아직 정하지 않은 내용이 있으면 그 부분만 질문합니다.

처음 만들 제품이나 새로 바꾸려는 내용은 사용자에게서 나와야 합니다. 그 내용은 인터뷰에서 이야기하고, 이후에는 달라진 요구나 여러 후보 중 작업할 대상이 있을 때만 덧붙입니다. 아래의 상세 조건과 결과물 예시는 읽고 확인하는 기준입니다.

3. 다음에 무엇을 할지 모르겠다면

```
/ask-matt 나 이제 뭐해야 해?
```

추천 이유를 읽고 추천받은 `/스킬이름` 을 실행하면 됩니다.

함께 읽기: [목적별 /archify 프롬프트 메뉴얼](#) — 전체 구조, 권한, 업무 흐름, 데이터, 보안, 변경 영향과 테스트를 시각화하는 12가지 요청 예시.

처음부터 모든 용어를 외울 필요는 없습니다. 이 문서는 **고등학교에서 사용할 공지사항 게시판을 친구와 함께 만든다**고 생각하며 읽으면 됩니다. 어려운 설명 아래에는 같은 뜻의 비유를 붙였습니다. 명령어는 비유로 바꾸지 않고 그대로 복사해 사용합니다.

처음 읽는 분: 이렇게 따라오세요

기본은 하나의 프로젝트 폴더, 하나의 대화에서 1강부터 6강까지 순서대로 진행하는 것입니다. 스킬 하나를 끝낼 때마다 새 폴더나 새 대화를 만들지 않습니다. 맨 아래 10개 워크플로우는 추가 예시이므로 지금 전부 실행하지 않아도 됩니다.

어디에 입력하나요?

문서에 보이는 내용	입력하거나 확인할 곳	예
<code>/grill-with-docs</code> 처럼 스킬로 시작하는 요청	프로젝트를 연 코딩 에이전트의 대화창	질문에 답하며 요구사항 정하기
<code>git</code> , <code>npm</code> , <code>npx</code> , <code>java</code> 로 시작하는 실행 명령	macOS 터미널 / Windows PowerShell	설치, 앱 실행, GitHub에 코드 올리기

어떻게 공부해야 하나요?

이 매뉴얼 저장소 `matt-user-guide` 는 **설명서**이고, 여러분이 만들 `notice-board` 는 **실습 프로젝트**입니다. 1강에서 `notice-board` 를 한 번 만들고 에이전트의 작업 폴더로 엽니다. 이후 파일은 그 안에 계속 쌓입니다.

- **처음부터 만들기:** 1강에서 빈 `notice-board` 를 만들고 시작합니다. 이 문서의 기본 경로입니다.
- **완성된 예제만 실행하기:** **실행 저장소 README**의 복제·실행 안내를 따릅니다. 이미 코드가 있으므로 생성 명령을 다시 실행하지 않습니다.

시작 전 준비

이 튜토리얼의 공통 필수조건은 코딩 에이전트, Git, Node.js(`npm·npx` 포함), Docker Desktop 설치 및 실행입니다. macOS에서는 Homebrew도 먼저 설치합니다. Docker Desktop은 Matt 스킬 자체의 실행 조건이 아니라, 이 실습에서 컨테이너를 사용하는 작업을 위한 준비 기준입니다.

macOS: 터미널에서 준비

Spotlight에서 **터미널**을 찾아 엽니다. 기본 셸(`zsh`)을 사용하면 되며 PowerShell을 설치할 필요는 없습니다.

1. **Homebrew 공식 안내**의 설치 명령을 터미널에서 실행합니다. 이미 설치했다면 `brew --version` 으로 확인하고 넘어갑니다.

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

설치 중 개발 도구 설치 안내가 나오면 따릅니다. 설치 마지막의 **Next steps**에 표시된 PATH 설정 명령도 실행한 뒤 터미널을 다시 엽니다. Apple Silicon과 Intel의 설치 경로가 다를 수 있으므로 본인 화면에 나온 명령을 사용합니다.

2. Homebrew를 확인하고 Git과 Node.js를 설치합니다. 이미 프로젝트에 맞는 버전을 사용 중이라면 중복 설치하지 않습니다.

```
brew --version  
brew install git node
```

3. [Docker Desktop macOS 설치 안내](#)에서 본인 Mac의 **Apple Silicon / Intel**에 맞는 설치 파일을 받습니다. 응용 프로그램 폴더에 설치하고 **Docker 앱을 실행**해 초기 설정을 마칩니다.

Windows: PowerShell에서 준비

[Git](#)과 [Node.js](#)를 설치한 뒤 PowerShell을 새로 엽니다. Windows에서는 Homebrew 설치 단계를 건너뜁니다.

[Docker Desktop Windows 설치 안내](#)에 따라 Docker Desktop을 설치합니다. WSL 2 백엔드를 사용하는 경우 안내된 WSL·가상화 요구사항도 충족해야 합니다. 설치 후 시작 메뉴에서 **Docker Desktop을 실행**하고 초기 설정을 마칩니다.

공통: 설치와 실행 확인

macOS 터미널과 Windows PowerShell에서 똑같이 실행할 수 있습니다.

```
git --version  
node --version  
npm --version  
npx --version  
docker --version  
docker compose version  
docker info
```

docker --version 만 성공하면 설치된 명령만 확인한 것입니다. **docker info** 에서 Server 정보까지 오류 없이 표시되어야 실행 준비가 된 것입니다. 연결 오류가 나면 Docker Desktop을 열어 엔진 시작이 끝났는지 확인합니다. 컨테이너를 사용하는 실습 중에는 Docker Desktop을 실행 상태로 유지합니다.

- ☐ 코딩 에이전트가 준비되어 있다.
- ☐ macOS에서는 `brew --version` 이 나온다.
- ☐ Git과 Node.js, npm·npx 명령이 동작한다.
- ☐ Docker Desktop을 설치하고 실행했다.
- ☐ `docker compose version` 과 `docker info` 가 정상 동작한다.

앱 구현에 필요한 추가 도구와 버전은 선택한 기술에 맞춰 에이전트의 안내를 따릅니다. 생성된 실행 스크립트도 운영체제에 맞춰 안내받습니다. `.ps1` 은 PowerShell용이므로 Mac 기본 터미널에서 그대로 실행하지 않습니다.

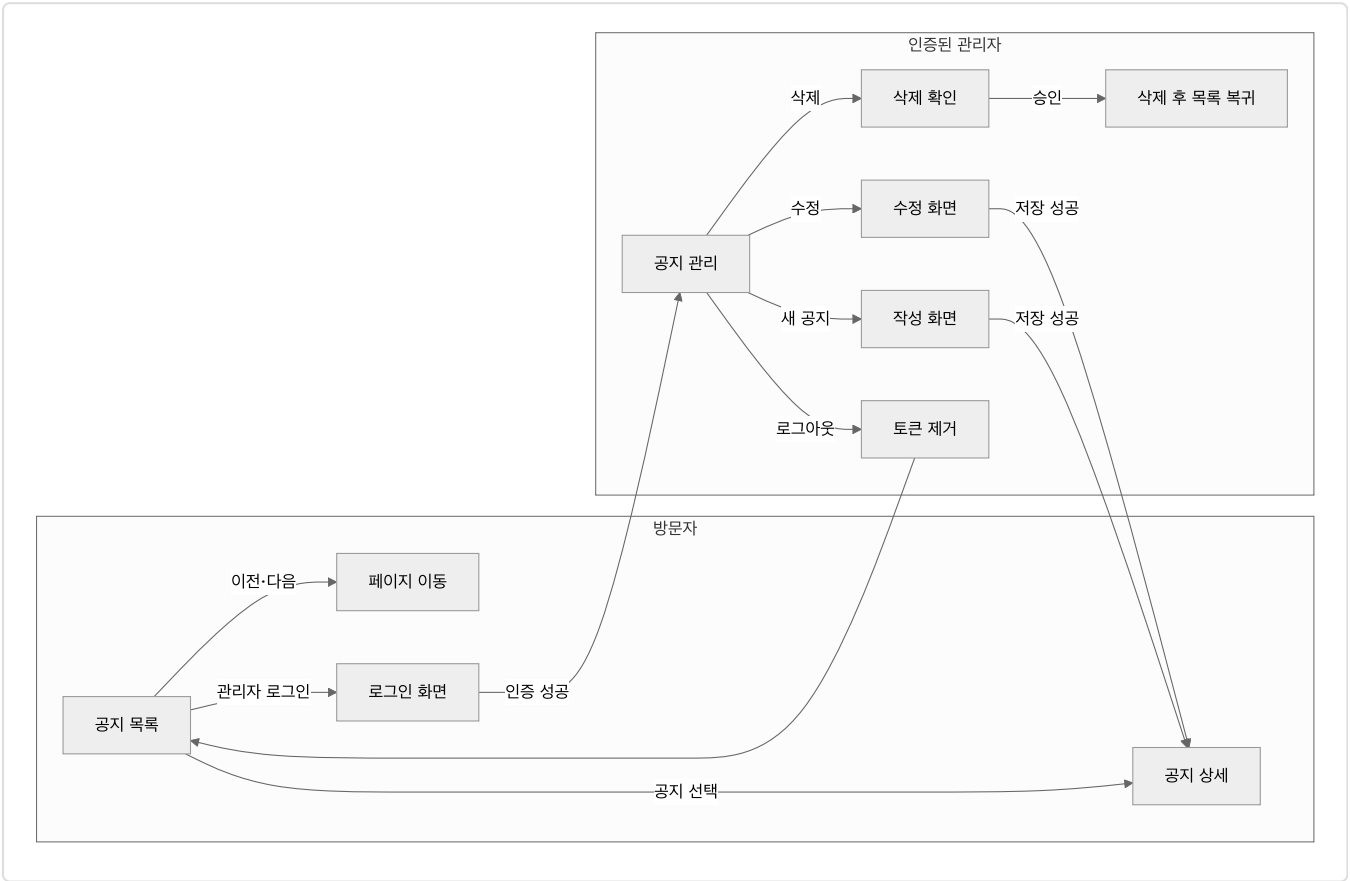
각 강은 **입력** → **예상 결과물** → **사용 전후 차이** → **완료 확인** 순서입니다. 대화와 결과물은 예시이며, 실제 실행 기록은 문서 마지막에 연결했습니다.

만들 제품: 공지사항 게시판

학생·학부모 등 방문자는 로그인 없이 고등학교의 공지 목록과 상세를 읽습니다. 학교 공지 담당자 한 명은 관리자로 로그인해 공지를 등록·수정·삭제합니다. 작성 즉시 공개되며, 초안·첨부파일·댓글·회원가입은 이번 범위에서 제외합니다.

이 기능 범위와 아래 세부 동작은 **튜토리얼의 제안값**입니다. 3강의 인터뷰에서 확인하고 확정하는 흐름으로 사용합니다.

아래는 화면의 배치가 아니라 **역할별 화면 이동과 가능한 동작**입니다. 쓰기 권한은 서버에서도 검사합니다.



실습 배경

예제는 Kotlin/Spring 백엔드와 React 프론트를 가진 게시판입니다. 테스트와 운영 배포도 작업 범위에 포함합니다. 기술 선택은 인터뷰에서 정할 조건이며, Matt 스킬은 다른 기술을 사용하는 프로젝트에도 적용할 수 있습니다.

이 가이드는 스킬을 언제 호출하고 어떤 결과를 확인하는지 설명합니다. 앱 설치·실행·환경변수·서비스별 배포 방법은 [실행 프로젝트 안내](#)를 참고합니다.

수업 지도

강	호출 / 작업	강이 끝나면 보이는 결과
1강	설치	필요한 스킬 목록
2강	하네스 세팅 — <code>/setup-matt-pocock-skills</code>	프로젝트 지침·설정 문서
3강	<code>/grill-with-docs</code>	합의된 동작, 용어집, 필요한 ADR
4강	<code>/to-spec</code>	기능·권한·DB·배포·검증 기준을 담은 명세
5강	<code>/to-tickets</code>	직접 실행해볼 작은 작업과 먼저 끝내야 할 작업
6강	<code>/implement</code>	앱 코드·테스트·워크플로우·배포 확인 기록

1강. 설치

설치 전에: 모델·하네스·스킬은 각각 무엇인가요?

모델은 글과 코드를 이해하고 다음 행동을 판단하는 AI입니다. ****하네스(harness)****는 모델이 파일을 읽고, 명령을 실행하고, 결과를 받아 다음 행동을 이어가도록 지원하는 실행 환경입니다. 대화 기록·도구 연결·권한 확인 같은 장치도 여기에 포함됩니다. 이 문서에서는 Claude Code나 Codex 같은 코딩 에이전트의 실행 환경을 설명할 때 이 말을 사용합니다.

기술실 비유: 모델은 문제를 생각하는 학생, 하네스는 작업대·공구·실습 규칙이 갖춰진 기술실입니다. 학생이 똑똑해도 사용할 공구가 없으면 나무를 자를 수 없습니다. 반대로 공구만 놓여 있다고 책상이 저절로 만들어지지도 않습니다.

스킬은 특정 작업을 수행하는 절차와 참고 자료를 묶어 재사용하는 작업 안내서입니다. 보통 `SKILL.md` 가 처음 읽는 문서이고, 필요하면 예제·템플릿·보조 스크립트를 함께 포함합니다. `/to-spec` 은 명세 작성법, `/archify` 는 구조도를 만드는 방법을 안내합니다. 스킬을 설치한다고 모델 자체를 다시 학습시키는 것은 아닙니다. [스킬 설명](#)

실습 비유: 기술실에서 ‘나무 의자 만들기 안내서’를 꺼내 읽는 것이 스킬 사용입니다. 책에는 준비물·작업 순서·확인 방법이 적혀 있습니다. 학생은 안내서를 읽고 실제 공구로 작업해야 합니다. 안내서만 설치했다고 의자가 완성되지는 않습니다.

용어	담당하는 일	게시판에서의 예
모델	이해하고 판단하기	로그인 오류의 설명과 코드를 해석
하네스	작업을 실행할 환경 제공	파일 읽기·터미널 실행·결과 전달
스킬	특정 작업의 진행 방법 안내	<code>/to-tickets</code> 로 명세를 작은 작업으로 분리
도구	구체적인 행동 실행	파일 저장, 테스트 실행, 배포 상태 조회

같은 스킬을 사용해도 연결된 도구·접근 권한·모델·프로젝트 상태에 따라 가능한 행동과 결과가 달라질 수 있습니다. / 스킬이름 이라는 호출 표기도 사용하는 도구에서 지원하는 방식으로 선택해야 합니다.

준비

만들 앱의 프로젝트 폴더를 코딩 에이전트에서 엽니다. 이 매뉴얼 저장소가 아니라 실습할 프로젝트가 작업 대상으로 선택되어 있는지 확인합니다. 같은 프로젝트에서 다음 강을 계속 진행합니다.

설치: 터미널 입력

```
npx --yes skills@latest add mattpocock/skills --global --skill "*" --agent codex claude-code --copy --yes
```

위 명령은 저장소의 **전체 스킬을 사용자 전역에 설치**하고, **Codex와 Claude Code 모두를 대상**으로 지정합니다. 확인 질문 없이 파일을 복사하므로 Claude Code의 기본 전역 경로인 `~/.claude/skills/` 에도 설치합니다. 공용 경로만 설치하고 끝내는 방식이 아닙니다. 아래 목록은 전체 설치 중 이 튜토리얼에서 주로 사용하는 스킬입니다. [스킬 설치 옵션](#)

옵션	뜻
<code>npx --yes</code>	npx의 패키지 실행 확인 생략
<code>--global</code>	현재 프로젝트가 아닌 사용자 전역 설치
<code>--skill "*"</code>	저장소에서 발견한 모든 스킬 설치
<code>--agent codex claude-code</code>	Codex와 Claude Code 모두 설치 대상으로 지정
<code>--copy</code>	에이전트 설치 위치에 다른 위치를 가리키는 링크 대신 실제 파일 복사
마지막 <code>--yes</code>	스킬 설치기의 확인 질문 생략

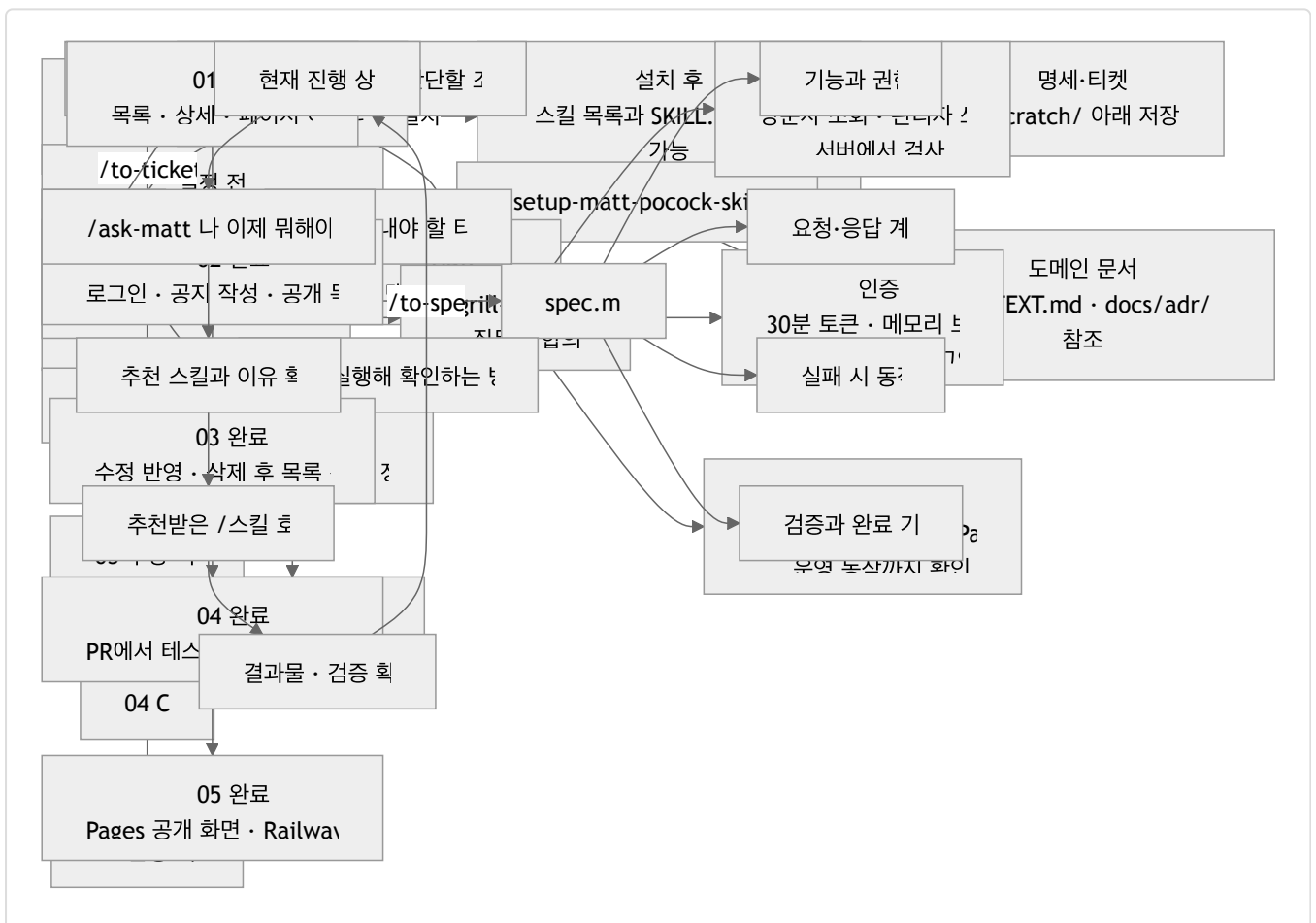
~ 는 사용자 홈 폴더입니다. macOS에서는 `/Users/사용자명` , 기본 설정의 Windows에서는 `C:\Users\사용자명` 에 해당합니다. `.agents/skills` 같은 공용 위치와 에이전트별 경로는 설치기 출력으로 확인하고, Claude Code의 `~/.claude/skills/` 도 확인합니다. 에이전트 경로를 별도로 설정한 환경에서는 실제 출력 경로를 따릅니다.

학교생활 비유: 필요한 준비물을 하나씩 고르는 대신 세트 전체를 준비하고, 내 개인 보관함에 넣어 어느 프로젝트에서도 꺼내 쓰게 합니다. Codex용과 Claude용 위치에 각각 복사하므로 두 도구에서 사용할 수 있습니다. 스킬은 전역 설치해도 `/setup-matt-pocock-skills` 로 정하는 프로젝트 규칙은 저장소마다 다릅니다.

스킬	역할
setup-matt-pocock-skills	저장소별 작업 규칙
grill-with-docs	질문과 문서화를 연결
grilling, domain-modeling	인터뷰, 용어집과 설계 결정 기록
to-spec, to-tickets	명세 작성, 구현 단위 분해
implement, tdd, code-review	구현, 검증, 리뷰
triage	들어온 요청을 분류하고 부족한 정보를 확인
ask-matt	현재 상황에 맞는 다음 스킬·진행 순서 추천

`grill-with-docs` 는 설치된 원문에서 `grilling` 과 `domain-modeling` 을 함께 호출합니다. 세 스킬이 모두 설치되어 있는지 확인합니다.

사용 전 → 후



설치 후에는 Claude Code와 Codex를 완전히 종료한 뒤 다시 실행하세요. 실행 중인 작업을 먼저 저장하고, 앱이나 터미널 세션을 종료한 뒤 다시 열어 스킬 목록을 확인합니다. 그래도 스킬이 보이지 않으면 설치 경로와 설치 성공 여부를 확인한 다음 PC를 재부팅하고 다시 확인하세요. 재부팅 후에도 보이지 않으면 설치 오류나 대상 경로 문제를 확인해야 합니다.

완료 확인: 설치기가 출력한 경로와 재시작한 에이전트의 스킬 목록에 필요한 스킬이 있는지 확인합니다. 아직 앱 파일은 생기지 않습니다.

2강. 하네스 세팅 — /setup-matt-pocock-skills

시작 상태: 스킬 설치와 재시작을 마쳤고, 빈 실습 폴더를 에이전트에서 열었습니다. **입력 위치:** 같은 프로젝트의 대화창.

끝나면: AGENTS.md, 이를 참조하는 CLAUDE.md와 작업 안내 파일이 생기고, GitHub 저장소 연결과 첫 커밋·푸시를 마칩니다. 아직 게시판 화면이 없는 것이 정상입니다.

실제 결과 보기 · 002: 작업 규칙이 없는 상태에서 AGENTS.md와 에이전트용 안내 문서가 생겼습니다. 무엇을 먼저 읽고 어디에 작업을 기록할지 확인해 보세요.

강별 링크의 002 - 006 은 수업 결과를 가리키는 태그입니다.

최초 1회만 호출하세요. /setup-matt-pocock-skills 는 프로젝트의 첫 작업 규칙을 만드는 스킬입니다. 이 프로젝트에서는 처음 단 한 번만 호출하고, 이후 작업에서는 반복하지 않습니다.

새 기능 추가, 새 대화 시작, Claude Code·Codex 재시작은 다시 호출할 이유가 아닙니다. 이미 만든 작업 규칙을 읽고 다음 작업을 이어가면 됩니다. **완전히 다른 프로젝트를 새로 시작할 때만 그 프로젝트에서 최초 1회 설정합니다.**

여기서 ‘하네스 세팅’은 코딩 에이전트가 이 프로젝트에서 일할 규칙을 설정한다는 뜻입니다. 프로그램 설치나 계정 로그인도 아니라, 프로젝트 지침과 이슈·문서 관리 규칙을 파일로 남기는 작업입니다.

쉬운 비유: 새 동아리방을 사용할 때 ‘회의록은 이 서랍, 할 일 카드는 저 게시판’이라고 처음에 정하는 것입니다. 모일 때마다 서랍 위치를 다시 정할 필요는 없습니다. 다른 동아리방으로 옮기거나 관리 방식을 바꿀 때만 다시 정하면 됩니다.

튜토리얼의 명세와 티켓은 로컬 Markdown으로 기록합니다. 코드는 GitHub에서 관리하고 CI/CD를 실행하면서, 작업 티켓은 저장소 안의 파일로 관리할 수 있습니다. GitHub Issues가 필수인 것은 아닙니다.

이슈를 GitHub에서 관리할까요, 파일로 관리할까요?

이슈는 해결할 문제나 구현할 작업을 기록한 것입니다. 이 흐름에서는 명세와 구현 티켓을 어디에 저장하고 찾아볼지 선택합니다.

비교	GitHub Issues	저장소 안의 Markdown 파일
저장 위치	GitHub의 이슈 페이지	<code>.scratch/<기능>/</code> 아래 파일
장점	여러 사람이 댓글·담당자·라벨로 협업하기 편함	에이전트가 코드와 함께 바로 읽고 수정하기 편함
변경 기록	이슈의 활동 기록으로 확인	Git에 포함하면 코드와 함께 diff·커밋으로 확인
외부 사용자 요청	이슈 링크를 공유해 접수하기 편함	파일 전달이나 저장소 변경 절차가 필요함
준비	GitHub 계정·저장소·접근 권한, 도구 인증 필요	파일을 읽고 쓸 수 있으면 바로 시작 가능
주의점	온라인 접근과 권한에 의존하고 코드와 이슈를 오가야 함	담당자·알림·상태 관리를 위한 별도 규칙이 필요함
협업 시 부담	이슈와 코드 변경의 연결을 유지해야 함	같은 파일을 수정하면 Git 충돌을 해결해야 할 수 있음

이 튜토리얼에서는 파일 관리를 더 추천합니다. 개인이나 작은 팀이 에이전트와 개발할 때는 명세·티켓·코드가 같은 작업 폴더에 있어 다음 작업에 전달하기 쉽고, 이슈를 다루기 위한 계정 연결 없이 시작할 수 있습니다. 특히 새 대화에서도 “이 명세와 티켓을 읽어줘”라고 파일 경로를 지정해 이어가기 편합니다.

파일을 쓴다고 GitHub를 사용하지 않는 것은 아닙니다. 명세와 티켓도 Git에 포함해 푸시하면 코드와 함께 공유할 수 있습니다. `.scratch/` 가 `.gitignore` 에 포함되어 있으면 업로드되지 않으므로 공유할 때 확인하세요. 파일 이름이나 폴더 이름만으로 자동 백업되는 것은 아닙니다.

쉬운 비유: GitHub Issues는 팀원들이 댓글과 담당자를 붙일 수 있는 온라인 업무 게시판입니다. 파일 관리는 작업물과 체크리스트를 같은 과제 폴더에 넣는 방식입니다. 혼자 또는 몇 명이 과제를 할 때는 같은 폴더에서 바로 확인하는 편이 간단합니다. 여러 반에서 요청을 받고 담당자를 배정해야 한다면 온라인 게시판이 더 편할 수 있습니다.

외부 사용자 요청이 많거나 담당자 배정·알림·여러 사람의 논의가 중요해지면 GitHub Issues를 고려하세요. 처음에는 파일로 시작하고 필요해졌을 때 전환해도 됩니다. 전환할 때는 기존 티켓과 새 이슈의 연결을 남겨 미완료 작업이 빠지지 않게 합니다.

학교 축제 비유: 일을 시작하기 전에 ‘회의록은 이 폴더, 할 일 카드는 저 폴더’라고 정하는 단계입니다. `/setup-matt-pocock-skills` 가 게시판을 만들어주는 것은 아닙니다. 앞으로 사용할 공책의 위치와 작성 규칙을 정해줍니다. `.scratch/` 라는 이름이어도 여기 담은 명세와 티켓은 다음 작업에 필요한 자료입니다.

입력

```
/setup-matt-pocock-skills
```

GITHUB 저장소 생성 후 연결해줘.
이슈와 명세는 로컬 Markdown.
triage는 기본 라벨을 사용.
AGENTS.md를 사용
CLAUDE.md는 단순히 AGENTS.md를 가리키도록.

예상 결과물

```
notice-board/
├── AGENTS.md
├── CLAUDE.md
└── docs/agents/
    ├── issue-tracker.md
    ├── triage-labels.md
    └── domain.md
```

이 실습에서는 위 요청대로 **작업 규칙의 본문은 AGENTS.md에만 작성**하고, CLAUDE.md는 그 파일을 참조하게 만듭니다. 기존 파일이 있어도 이 기준에 맞춰 중복 규칙을 정리합니다.

AGENTS.md와 CLAUDE.md — 언제 어떤 파일을 쓰나요?

둘 다 코딩 에이전트에게 프로젝트의 작업 규칙을 전달하는 Markdown 파일입니다. 어떤 도구가 읽는지가 다릅니다.

상황	사용할 파일	이 실습의 방식
Codex로 작업	AGENTS.md	문서 위치·검증 명령·작업 규칙을 여기에 작성
Claude Code로 작업	CLAUDE.md	아래 참조 한 줄로 같은 AGENTS.md를 읽게 함
두 도구를 번갈아 사용	두 파일	규칙 본문은 AGENTS.md 한 곳에서 관리

Codex는 작업을 시작할 때 AGENTS.md 지침을 읽습니다. [OpenAI 공식 안내](#)

Claude Code는 CLAUDE.md에서 @파일경로로 다른 파일을 불러올 수 있습니다. 따라서 프로젝트 루트의 CLAUDE.md 내용은 다음 한 줄이면 됩니다. [Claude Code 공식 안내](#)

```
@AGENTS.md
```

위 상자는 **파일에 넣을 내용**입니다. 실제 CLAUDE.md에는 코드 블록 기호 없이 @AGENTS.md 한 줄만 저장합니다. 두 파일은 같은 폴더에 둡니다. 규칙을 바꿀 때는 AGENTS.md를 수정합니다.

첫 커밋부터 두 지침 파일 포함하기

2장의 설정 요청과 함께 다음 조건도 전달합니다. **첫 커밋 전에 AGENTS.md와 CLAUDE.md를 모두 생성**해야 합니다.

```
GitHub 저장소는 README 자동 생성 없이 빈 저장소로 만들어 연결해줘.
첫 커밋 전에 AGENTS.md와, 내용이 @AGENTS.md 한 줄인 CLAUDE.md를 생성해줘.
설정 문서와 참조를 확인하고 두 파일을 첫 커밋에 포함해 바로 푸시해줘.
이후에도 작업을 완료하고 검증하면 바로 커밋하고 푸시해줘.
```

GitHub에서 README를 자동 생성하면 그 커밋이 먼저 생기므로 빈 원격 저장소로 시작합니다. GitHub 인증이나 저장소 소유자·이름이 아직 정해지지 않았다면 에이전트의 안내에 따라 정합니다. 예제 저장소 주소는 참고용이며, 각자의 실습 저장소는 본인 계정에 생성합니다.

첫 커밋의 파일 목록은 터미널에서 확인할 수 있습니다.

```
git log --reverse --oneline
git ls-tree -r --name-only HEAD
```

두 번째 명령은 첫 커밋 직후 실행합니다. 목록에 AGENTS.md 와 CLAUDE.md 가 모두 있어야 합니다. 이후에는 첫 번째 명령에서 확인한 최초 커밋 ID를 HEAD 대신 넣어 확인합니다.

학교생활 비유: 과학실 문에 붙은 ‘실험 전에 준비물을 확인하고, 끝나면 결과를 기록하세요’라는 안내판입니다. 실험마다 길게 다시 설명하지 않도록 공통 규칙을 적어둡니다. 하지만 안내판에 ‘불을 끄세요’라고 써놓는 것만으로 전등이 꺼지지 않는듯, 실제 실행과 검증은 별도입니다.

파일	쉽게 말하면	적합한 내용
AGENTS.md	여기서 일하는 공통 규칙	어떤 문서를 읽고 어떻게 검증할지
CLAUDE.md	Claude Code가 같은 규칙을 읽는 입구	이 실습에서는 @AGENTS.md 한 줄
SKILL.md	특정 작업의 안내서	명세 작성·진단·구조도 생성 절차
CONTEXT.md	우리 팀의 용어 사전	공지·관리자·삭제의 정확한 뜻
ADR	중요한 결정의 이유 노트	왜 이 인증·배포 방식을 골랐는지

축제 준비 비유: ‘끝나면 정리한다’는 전체 규칙, ‘음료를 만드는 순서’는 작업 안내서, ‘운영요원이 누구인지’는 용어 사전, ‘왜 실내 부스를 골랐는지’는 결정 기록입니다. 네 문서는 서로 돕지만 담는 내용은 다릅니다.

파일	확정되는 내용
프로젝트 지침	어떤 상황에 설정 문서를 읽어야 하는지
issue-tracker.md	.scratch/<기능>/spec.md , issues/<번호>-<이름>.md
triage-labels.md	needs-triage , needs-info , ready-for-agent , ready-for-human , wontfix
domain.md	용어집·ADR의 위치와 읽는 규칙

사용 전 → 후

완료 확인: 설정 문서 세 개와 `AGENTS.md` 의 참조가 일치하고, `CLAUDE.md` 가 `AGENTS.md` 를 불러오는지 확인합니다. 두 지침 파일이 첫 커밋과 원격 저장소에 포함되어 있어야 합니다. `CONTEXT.md` 자체는 의미 있는 용어가 정리되는 다음 강에서 생성합니다.

3강. /grill-with-docs — 원하는 기능을 질문과 답변으로 분명하게 정하기

시작 상태: 2강의 작업 규칙 파일이 있습니다. **입력 위치:** 그대로 같은 대화창. **할 일:** 에이전트의 질문에 답하고 합의 내용을 확인합니다. **끝나면:** 용어와 결정 이유가 문서에 남습니다. 아직 앱을 실행하는 단계는 아닙니다.

실제 결과 보기 · 003: 질문과 답변으로 정한 내용을 `CONTEXT.md`와 `ADR 결정 기록`에 남겼습니다. 막연한 “공지 게시판”이 어떤 용어와 규칙을 가진 제품으로 구체화됐는지 확인해 보세요. [이전 단계와 비교](#)

이번에 추가한 핵심 단계입니다. ‘시큐리티를 쓴다’는 말만으로는 누가 무엇을 할 수 있는지, 로그인은 얼마나 유지되는지 결정되지 않습니다. 구현 전에 실제 사용 상황을 질문하고 답을 문서로 남깁니다.

학교 축제 비유: ‘매점 만들어줘’라고만 하면 친구는 음료만 팔 생각을 하고, 나는 떡볶이도 팔 생각을 할 수 있습니다. 만들기 전에 ‘무엇을 팔까? 누가 돈을 받을까? 환불은 가능할까?’를 묻는 것이 그릴워드독스입니다. 귀찮게 시험하는 게 아니라, 서로 다른 완성을 상상하고 있지 않은지 확인하는 과정입니다.

입력

/grill-with-docs

공지사항 게시판을 만들고 싶어.

방문자는 목록과 상세를 보고, 관리자 한 명만 등록·수정·삭제해.

회원가입, 댓글, 첨부파일, 검색, 초안, 상단 고정은 이번 범위에서 빼자.

백엔드(backend/)는 최신 안정 Spring Boot + Kotlin, JPA, Spring Security야.

로컬 DB는 H2 파일, 테스트 DB는 H2 인메모리, 운영 DB는 PostgreSQL.

백엔드는 Railway, 프론트는 Vite React TypeScript shadcn/ui야.

프론트(frontend/)는 Cloudflare Pages로 배포하고 GitHub Actions가 CI/CD를 담당해.

백엔드 폴더는 backend/, 프론트엔드 폴더는 frontend/로 생각하고 있어.

같은 저장소에 둘 때 각 폴더의 역할과 실행·배포 경로를 질문으로 확인하고 합의해줘.

도메인은 공지사항 하나이므로 루트의 CONTEXT.md와 docs/adr/를 공유하고 싶어.

권한, 인증 유지, 입력 제한, 오류,

목록을 여러 페이지로 나누고 이동하는 방법, DB 전환, 자동배포 완료 기준을 질문해서 구체적으로 정해줘.

버전·플랫폼 제약은 공식 문서를 직접 확인해줘.

예상 인터뷰: 한 번에 모든 답을 추측하지 않기

라운 드	에이전트가 확인할 질문 예시	튜토리얼에서 선택할 답
1	백엔드·프론트 폴더 이름과 공통 문서 위치는 어떻게 할까요?	backend/ , frontend/ ; 루트의 CONTEXT.md 와 docs/adr/ 공유
1	공지는 저장 즉시 공개되나요? 삭제는 복구되나요?	즉시 공개, 확인 후 영구 삭제
1	제목·본문 제한과 정렬 기준은 무엇인가요?	제목 1-100자, 본문 1-10,000자; 앞뒤 공백 제거; 최신순
1	한 페이지 크기와 빈 화면은 어떻게 하나요?	10개 고정, 없으면 안내; 마지막 항목 삭제 시 이전 페이지로
2	관리자 로그인을 새로고침 후에도 유지해야 하나요?	실습에서는 다시 로그인해도 됨
2	프론트·API 주소가 다를 때 인증은 어떻게 할까요?	짧은 Bearer 토큰, 쿠키 인증 없이 메모리에 보관
3	토큰 만료·로그아웃은 어떤 의미인가요?	30분 만료, refresh 없음; 로그아웃은 브라우저 토큰 제거
3	테스트는 어디에 입력하고 무엇을 확인할까요?	Security 포함 HTTP 통합 테스트 + H2 메모리, 사용자 화면 테스트, 실제 백엔드 +H2 브라우저 E2E; 운영 PG 검증은 별도
3	CI와 실제 서비스 확인 중 어디까지가 완료인가요?	검증 통과, 배포된 커밋 확인, 운영 화면·DB 저장 확인

실제 질문은 앞선 답에 따라 달라집니다. 예를 들어 토큰 방식을 선택하기 전에는 토큰 만료 시간을 확정하지 않습니다.

질문에 답해서 정해진 조건

주제	결정 예시
공지	제목·일반 텍스트 본문·생성일·수정일; HTML을 실행하지 않음
목록	생성일 내림차순, 동물은 ID 내림차순, 10개씩 페이지 이동
작성	관리자만 가능; 저장 성공 후 상세 화면으로 이동
수정	생성일과 목록 순서는 유지; 수정일 갱신
삭제	확인창에서 승인 후 삭제, 목록 복귀; 없는 공지는 404
오류	입력 오류 400, 인증 오류 401, 권한 부족 403, 대상 없음 404
관리자	담당자 한 명만 로그인; 회원가입 없음
인증	앱이 로그인 성공 시 서명한 access JWT 발급; Spring Security로 검증
토큰	30분, 프론트 메모리에만 저장; 새로고침·만료 시 재로그인
로그아웃	현재 화면에서 토큰 제거; 이미 발급된 토큰은 만료 전까지 유효
운영 연결	허용된 프론트 Origin만 CORS 허용; Authorization 헤더 허용

실제 결과물: 용어집과 결정 기록

실습 저장소의 3강 태그 **003**, 커밋 **fc147a7** (docs: define notice board requirements and decisions)에서 아래 세 파일이 추가되었습니다. 다음 내용은 해당 커밋의 원문 전체입니다.

```
이 커밋에서 추가된 문서
├─ CONTEXT.md
├─ docs/adr/
│   ├── 0001-admin-authentication.md
│   └─ 0002-ci-controlled-deployment.md
```

CONTEXT.md 원문:

```
# 공지사항

방문자가 안내를 읽고 관리자가 게시하는 공간이다.

## Language

**공지(Notice)**: 관리자가 등록하면 즉시 공개되는 제목과 일반 텍스트 본문을 가진 안내문.
**방문자(Visitor)**: 로그인 없이 공지를 읽는 사람.
**관리자(Administrator)**: 공지를 등록·수정·삭제하는 담당자.
**삭제>Delete)**: 공지를 영구 제거하는 동작. 숨김이나 휴지통을 뜻하지 않는다.
```

docs/adr/0001-admin-authentication.md 원문:

메모리에 보관하는 짧은 Bearer 토큰

관리자 한 명과 공개 조회가 필요한 실습이다. 서로 다른 서비스 도메인을 사용하므로 쿠키 세션 대신 30분 access JWT를 프론트 메모리에 보관한다. 새로고침하면 재로그인하며 로그아웃은 브라우저 토큰 제거일 뿐 서버의 즉시 폐기가 아니다. 관리자명과 BCrypt 해시 및 충분히 긴 서명 키는 운영 환경에 주입한다. Refresh token, 계정 관리, 외부 인증 제공자는 범위 밖이다.

[docs/adr/0002-ci-controlled-deployment.md](#) 원문:

검증한 커밋만 배포

GitHub Actions에서 H2 HTTP 테스트, 화면 테스트, E2E를 통과한 main 커밋을 Railway와 Pages에 배포한다. 서비스 Git 자동배포는 사용하지 않아 검증을 건너뛰는 경로를 피한다. 두 서비스는 원자적 배포가 아니므로 API의 기존 호출을 유지하고 배포 ID와 커밋을 별도로 확인한다. H2 통과를 PG 호환성 증거로 사용하지 않는다.

이 기록은 당시 합의한 용어와 설계 결정입니다. 배포 성공을 증명하는 실행 결과는 아닙니다.

CONTEXT.md — 우리 팀이 같은 뜻으로 말하기 위한 작은 사전

CONTEXT.md 는 이 스킬 흐름에서 프로젝트의 업무 용어를 정의하는 파일입니다. 이름만 보면 모든 배경 지식을 넣어야 할 것 같지만, 여기서는 용어집으로 사용합니다. 해야 할 일 목록이나 모든 대화를 붙여 넣는 파일이 아닙니다.

예를 들어 친구는 ‘관리자’를 학교 선생님 전체라고 생각하고, 나는 게시판 담당자 한 명이라고 생각할 수 있습니다. 이 상태로 ‘관리자만 삭제 가능’이라고 쓰면 서로 다른 기능을 만들게 됩니다.

학교생활 비유: 조별 발표에서 ‘자료 담당’이라는 말을 먼저 정의하는 것과 같습니다. 한 명은 사진만 모으는 역할이라고 생각하고 다른 한 명은 발표 자료까지 만드는 역할이라고 생각하면 일이 빠집니다. ‘자료 담당 = 출처를 확인해 글과 사진을 모으는 사람’이라고 합의해두면 같은 뜻으로 이야기할 수 있습니다.

게시판의 용어집은 다음처럼 짧고 구체적으로 씁니다.

- 공지: 관리자가 등록하고 방문자가 읽는 안내문. 등록하면 바로 공개된다.
- 방문자: 로그인하지 않고 공개 공지를 읽는 사람.
- 관리자: 이 게시판의 공지를 등록·수정·삭제하는 담당자.
- 공지 삭제: 공지를 영구 제거하는 동작. 임시 숨김을 뜻하지 않는다.

언제 읽나요? 새 기능을 논의하거나 코드를 살펴보기 전에 읽습니다. **언제 고치나요?** 인터뷰에서 단어의 뜻을 합의했을 때 고칩니다. 아직 정하지 않은 뜻을 확정된 것처럼 쓰지 않습니다.

ADR — 중요한 선택의 ‘왜’를 남기는 결정 노트

ADR은 **Architecture Decision Record**, 즉 중요한 설계 결정을 기록하는 문서입니다. docs/adr/ 는 그런 기록을 모아두는 폴더입니다. 보통 결정 하나를 파일 하나에 씁니다.

CONTEXT.md 가 ‘이 말은 무슨 뜻인가?’에 답한다면, ADR은 ‘**여러 방법 중 왜 이 방법을 골랐는가?**’에 답합니다.

학교 축제 비유: 매점을 운동장이 아니라 체육관에 열기로 했다고 합시다. ‘체육관 사용’만 쓰면 나중에 친구가 ‘운동장이 더 넓은데 왜?’라고 묻습니다. ‘비 예보가 있고, 체육관에는 전기가 있으며, 대신 자리가 좁아진다’까지 적어두면 선택의 이유와 불편함을 함께 이해할 수 있습니다. 이것이 ADR의 역할입니다.

이 게시판에서는 로그인 방법이 좋은 예입니다. 아래의 두 방법은 각각 얻는 것과 감수할 것이 다릅니다.

선택지	얻는 것	고려할 점
메모리에 짧은 토큰 보관	서로 다른 서비스 주소에서도 쿠키 인증 없이 요청 가능	새로그침 후 재로그인, 즉시 토큰 폐기는 별도 설계 필요
쿠키 세션	브라우저의 세션 쿠키로 로그인 유지 가능	서비스 도메인·쿠키 정책·CSRF·세션 저장을 함께 설계해야 함

‘메모리 토큰이 항상 최고’여서 고르는 것이 아닙니다. **관리자 한 명이 사용하는 작은 실습이며 재로그인을 허용한다는 조건에서** 선택하는 것입니다. 그 조건이 달라지면 결정도 다시 검토할 수 있습니다.

ADR을 만들기 좋은 때는 선택을 바꾸는 비용이 크고, 나중에 이유를 궁금해할 만하며, 실제로 비교한 대안이 있을 때입니다. 버튼 글자를 ‘저장’에서 ‘등록’으로 바꾸는 정도마다 ADR을 만들 필요는 없습니다.

인증 ADR 일부 내용 예시:

관리자 인증 방식

상태: 수락

맥락: 서로 다른 서비스 도메인에서 운영하며 관리자 한 명만 사용한다.

결정: 짧은 Bearer 토큰을 프론트 메모리에 보관한다.

대안: 쿠키 세션, 외부 인증 제공자.

결과: 새로그침 시 재로그인한다. 즉시 토큰 폐기는 이번 범위에 없다.

결정을 바꿀 때는 예전 이유를 지워버리기보다 새 결정을 기록하고 이전 기록과 연결합니다. ‘예전에는 틀렸다’보다 ‘사용 조건이 바뀌어 이제 다른 방법을 선택했다’는 과정을 남기는 것입니다.

CONTEXT.md·ADR·명세·티켓은 무엇이 다른가요?

Matt 스킵 흐름의 큰 장점은 개발하면서 프로젝트를 이해하는 데 필요한 기록도 함께 쌓인다는 것입니다. `/grill-with-docs` 에서 뜻을 합의하고 선택의 이유를 남기면, 다음 작업에서 LLM이 그 기록을 읽고 우리 프로젝트에 맞춰 판단할 수 있습니다. 기록이 정확하게 유지될수록 같은 설명을 반복할 필요가 줄고, 세밀한 규칙을 놓칠 가능성도 줄어듭니다.

조별 과제 비유: 새 친구가 조에 들어올 때마다 “우리는 왜 이 주제를 골랐고, 이 단어를 무슨 뜻으로 쓰는지” 처음부터 설명하면 빠지는 이야기가 생깁니다. 그런데 용어 사전과 결정 노트가 있다면 새 친구도 읽고 이어서 작업할 수 있습니다. 과제를 진행하며 기록을 보강할수록 다음에 참여한 친구도 우리 조의 사정을 더 깊이 이해하게 됩니다.

예를 들어 몇 주 뒤 “로그인을 편하게 바꿔줘”라고 요청했다고 해봅시다.

기록이 없다면 놓치기 쉬운 점	기록을 읽으면 확인할 수 있는 점
관리자가 여러 명이라고 추측	CONTEXT.md에서 이 게시판의 관리자 역할 확인
새로그침 후 로그인 해제를 단순 버그로 판단	ADR에서 재로그인을 허용한 이유와 조건 확인
‘삭제’를 임시 숨김으로 구현	CONTEXT.md와 명세에서 영구 삭제라는 합의 확인
편의 기능을 고치면서 기존 권한 규칙을 변경	명세·티켓·테스트에서 지켜야 할 동작 확인

중요한 것은 LLM 자체가 프로젝트를 영구 학습하는 것이 아니라, 다음 작업에서도 읽을 수 있는 ‘프로젝트의 기억’을 파일로 남긴다는 점입니다. 새 대화나 다른 에이전트에서도 같은 문서를 읽으면 맥락을 복원하기 쉬워집니다. 문서를 읽지 않거나 기록이 오래되어 실제 코드와 다르면 여전히 놓치거나 잘못 판단할 수 있으므로, 합의가 바뀌면 문서도 갱신하고 결과는 테스트로 확인합니다.

시험공부 비유: 머릿속 기억만 믿는 대신 오답 노트에 “어디서 헛갈렸고 왜 이 풀이를 골랐는지” 적어두는 것입니다. 노트가 있다고 절대 틀리지 않는 것은 아니지만, 다음 문제를 풀기 전에 읽으면 같은 실수를 줄일 수 있습니다.

같은 대화에서 작업을 이어갈 때는 다음 스킬만 호출합니다. 관련 문서를 읽고 기존 합의를 이어받는 것은 에이전트가 할 일입니다. 새 대화에서 대상이 불분명할 때만 작업할 기능이나 명세 경로를 짧게 알려줍니다.

모든 세부 사항을 CONTEXT.md와 ADR에 몰아넣지는 않습니다. 용어는 CONTEXT.md, 중요한 선택의 이유는 ADR, 구체적인 동작은 명세·티켓, 실제 보장은 코드·테스트에 나누어 남깁니다. 이렇게 연결된 기록이 프로젝트 이해를 점점 풍부하게 만듭니다.

문서	답하는 질문	학교 축제에 비유하면	게시판 예시
CONTEXT.md	이 단어는 무슨 뜻인가?	조별 활동 용어 사전	관리자는 공지를 관리하는 담당자
ADR	왜 이 방법을 선택했나?	체육관을 고른 이유 기록	재로그인을 허용하고 메모리 토큰 선택
spec.md	무엇이 되면 완성인가?	부스의 완성 조건	관리자가 공지를 등록하면 방문자가 읽을 수 있음
티켓	이번에 끝낼 작은 일은 무엇인가?	오늘 할 일 카드	로그인부터 공지 등록까지 구현·검증

사용 전 → 후

완료 확인: 에이전트의 최종 합의 요약을 읽고 “이 조건으로 합의했어. 문서에 반영해줘”라고 답합니다. 아직 앱 구현은 시작하지 않습니다.

4강. /to-spec — 함께 정한 기능과 완성 조건을 문서로 남기기

시작 상태: 3강에서 기능과 규칙을 합의했습니다. **할 일:** 같은 대화에서 명세 작성을 요청하고 빠진 기능이 없는지 읽습니다. **끝나면:** `.scratch/notice-board/spec.md` 가 생깁니다. `.scratch` 를 포함한 경로는 에이전트가 만들게 하면 됩니다.

실제 결과 보기 · 004 커밋: 앞에서 정한 규칙이 구현할 기능과 확인 조건으로 정리된 파일입니다. 아래 입력 예시와 실제 명세를 나란히 읽어 보세요. [이전 단계와 비교](#)

이렇게 무엇을 만들고 어떤 조건을 만족해야 하는지 적은 문서를 **명세**라고 부릅니다.

`/grill-with-docs` 에서 결정한 내용을 `/to-spec` 으로 모아 정리합니다. 이 단계는 새 요구사항 인터뷰보다 합의된 내용을 빠짐없이 옮기는 데 집중합니다. 설치된 스킴은 테스트에서 입력과 결과를 확인할 위치를 사용자에게 확인하는 단계도 포함합니다.

학교 축제 비유: 인터뷰가 ‘어떤 매점을 만들지 회의하는 시간’이라면 명세는 ‘음료 3종을 팔고, 가격표가 있으며, 현금 결제가 되면 완성’이라고 쓴 합의서입니다. 완성 조건을 먼저 써야 나중에 ‘이 정도면 끝난 거 아니야?’라는 다툼을 줄일 수 있습니다.

테스트에서 입력과 결과를 확인할 위치란? 어디에서 입력하고 어디에서 결과를 확인할지 정하는 것입니다. 자판기를 검사할 때 나사 모양부터 보는 대신 ‘돈을 넣고 버튼을 누르면 맞는 음료가 나오는가’를 확인하는 것과 같습니다. 이 실습에서는 API 요청·응답과 화면 조작을 주요 확인 지점으로 삼습니다.

테스트 설명에서 나오는 말

통합 테스트는 여러 부분을 연결해 함께 확인하는 테스트입니다. 브라우저 E2E 테스트는 사람이 화면에서 클릭하듯 실제 화면부터 서버 저장까지 끝에서 끝으로 확인합니다. CRUD는 만들기·읽기·수정·삭제를 묶어 부르는 말입니다.

쉬운 비유: 버튼만 눌러는지 보는 것과, 주문 버튼을 누른 뒤 주방에 주문이 들어가고 결과가 화면에 보이는지 확인하는 것은 다릅니다. E2E는 이 연결된 과정을 따라가 보는 것입니다.

입력

/to-spec

이 한 줄로 앞서 합의한 기능, API 요청·응답, 관리자 인증, 환경별 DB, CI/CD 완료 기준을 명세로 정리하는 단계에 들어갑니다. 문서 위치는 2강의 설정을 따릅니다. 명세가 구현 파일 목록이나 코드로 대체되지 않았는지 결과를 확인합니다.

테스트 확인 지점을 질문받으면 제안을 읽고 답합니다. 이미 합의한 조건을 다시 작성할 필요는 없습니다. 이 실습에서 확인할 기준은 다음과 같습니다.

검증	입력하는 곳	확인할 결과
백엔드 HTTP 통합 테스트	Security 필터를 통과하는 HTTP 요청, H2 인메모리 DB	상태 코드·응답·저장 결과·인증 및 권한 거부
프론트 사용자 화면 테스트	로그인·공지 작성 등의 화면 조작	표시 내용·오류 안내·화면 이동
브라우저 E2E	실제 백엔드와 H2에 연결한 브라우저	화면 조작부터 서버 저장·재조회까지 연결된 동작
별도 운영 배포 검증	배포된 서비스와 PostgreSQL	마이그레이션 적용·실제 저장·배포 커밋과 서비스 준비 상태

H2 테스트 통과가 운영 PostgreSQL 검증을 대신하지 않습니다.

결과물 A: 무엇을 만들지 적은 문서의 내용

```
# 공지사항 게시판
Status: ready-for-agent

## Problem Statement
방문자는 최신 안내를 읽고 관리자는 개발자 도움 없이 안내를 관리해야 한다.

## Solution
공개 읽기 화면과 관리자 쓰기 기능을 가진 게시판을 제공한다.

## User Stories
1. 방문자는 최신순 목록에서 공지를 선택할 수 있다.
2. 방문자는 제목과 본문, 작성·수정 시점을 확인할 수 있다.
3. 관리자는 로그인해 공지를 등록할 수 있다.
4. 관리자는 내용을 수정하고 삭제할 수 있다.
5. 관리자는 입력 오류와 인증 만료의 원인을 확인할 수 있다.
6. 운영자는 검증된 main 변경을 자동 배포할 수 있다.

## Implementation Decisions
합의한 권한·API·저장·환경·배포할 때 지킬 약속을 기록한다.

## Testing Decisions
HTTP/H2, 화면, H2 E2E, 운영 PG 검증의 범위를 구분한다.

## Out of Scope
회원가입, 댓글, 첨부, 검색, 초안, 상단 고정, refresh token.

## Further Notes
버전과 외부 서비스의 실제 설정값은 구현·배포 문서에 기록한다.
```

일부 내용 예시입니다. 실제 명세는 페이지 끝 삭제, 만료, 중복 제출, 없는 공지, 실패 중 입력 보존 등 합의한 사례까지 사용자 스토리와 검증 조건으로 포함합니다.

명세에서 확인할 것

- 합의한 기능과 제외 범위가 담겼는가?
- 요청·응답, 권한, 실패 상황의 동작이 분명한가?
- 테스트에서 무엇을 입력하고 어떤 결과를 확인할지 정했는가?
- 테스트 통과와 운영 배포 확인을 구분했는가?

구체적인 API와 DB 설계는 프로젝트 명세에서 확인합니다.

사용 전 → 후

완료 확인: 대화의 결정과 명세를 대조합니다. H2 파일/메모리/PG의 구분, 쓰기 권한, 배포 완료 조건이 빠지지 않아야 합니다.

5강. /to-tickets — 하나씩 만들고 직접 실행해볼 수 있는 작은 작업으로 나누기

시작 상태: 4강의 spec.md를 확인했습니다. **할 일:** 제안한 작업 순서를 읽고 승인합니다. **끝나면:** `.scratch/notice-board/issues/` 에 작업별 Markdown 파일이 생깁니다. 여기서 티켓은 유료 이용권이 아니라 ‘할 일 한 장’입니다.

실제 결과 보기 · 005 커밋: 하나의 명세가 조회, 로그인·작성, 수정·삭제, CI, 운영 배포 작업으로 나뉩니다. [첫 번째 티켓](#)을 열면 실제 작업 크기를 볼 수 있습니다. [이전 단계와 비교](#)

티켓은 이번에 끝낼 작은 작업과 확인 방법을 적은 카드입니다. 먼저 끝내야 할 티켓은 해당 작업을 시작하기 전에 끝나야 하는 일입니다.

학교 축제 비유: ‘매점 완성’은 너무 큼니다. ‘손님이 음료를 고르고 돈을 내면 음료를 받는다’처럼 실제로 해보고 결과를 확인할 수 있는 작은 일로 나눕니다. ‘예쁜 간판만 만들기’처럼 한 부분만 끝내는 것보다 실제 사용 경로를 하나씩 완성하는 방식입니다. 포스터를 붙이려면 먼저 문구가 정해져야 하는 것처럼 작업에도 선후 관계가 있습니다.

입력

/to-tickets

명세와 프로젝트 설정을 이어받아 작은 기능 단위, 선후 관계, 완료 조건, 직접 확인할 방법을 제안합니다. 사용자가 이 작성 절차를 호출 뒤에 다시 적을 필요는 없습니다. CI와 운영 배포도 합의한 명세 범위에 따라 포함합니다.

예상 분할

번호	티켓	먼저 끝낼 작업	완료 후 눈으로 볼 결과
01	공개 목록·상세 읽기	없음	local 전용 예제 공지를 UI에서 조회, H2 API 테스트 통과
02	관리자 로그인·공지 등록	01	로그인 후 작성, 새 공지가 공개 목록에 표시
03	공지 수정·삭제	02	본문 변경 반영, 삭제 후 목록에서 사라짐
04	코드를 합치기 전 자동 검사	03	GitHub PR에서 백엔드·프론트·E2E 검사 결과 표시
05	운영 DB·자동배포	04	Railway API + PG, Pages UI, main 자동배포 결과

01에 프로젝트 초기 구성과 shadcn/ui 설정도 포함합니다. 읽기 기능부터 연결하며, 예제 데이터는 local 프로필 전용으로 운영에 들어가지 않습니다. 04·05는 서버와 자동 검사·배포를 준비하는 티켓이지만 완료 조건이 ‘파일 생성’이 아니라 실제로 확인할 수 있는 검사·배포 결과입니다. 작은 실습이라 앞 작업을 끝내고 다음 작업을 하는 순서를 택한 것이며 항상 순차로 분해해야 하는 것은 아닙니다.

제안된 분할과 순서를 확인하고, 맞으면 짧게 동의합니다. 저장 위치와 티켓 작성 규칙은 프로젝트 설정을 따릅니다.

예상 파일 트리

```
.scratch/notice-board/  
├── spec.md  
└── issues/  
    ├── 01-read-notices.md  
    ├── 02-login-and-create.md  
    ├── 03-update-and-delete.md  
    ├── 04-ci.md  
    └── 05-production-deploy.md
```

티켓 일부 내용: 등록 기능

02: 관리자 로그인과 공지 등록

****What to build:**** 관리자가 로그인해 공지를 작성하면 방문자가 읽을 수 있다.

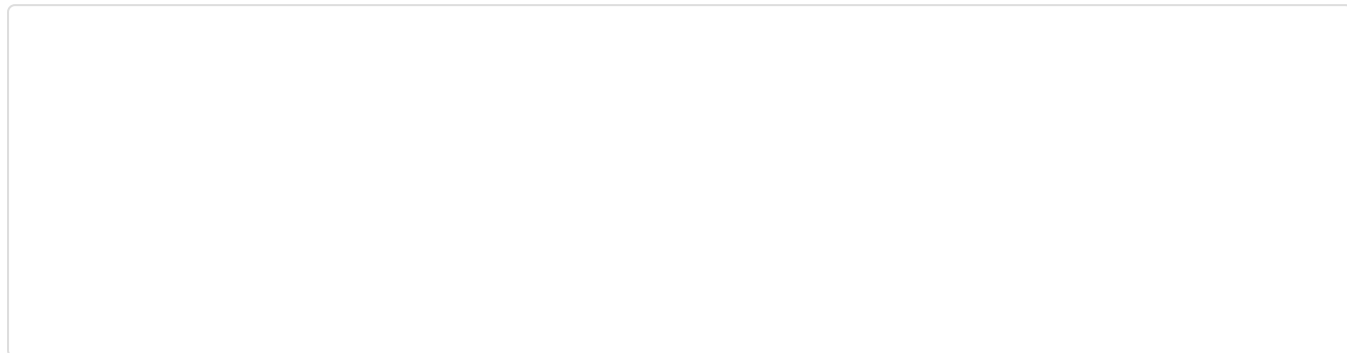
****Blocked by:**** 01: 공개 목록·상세 읽기

****Status:**** ready-for-agent

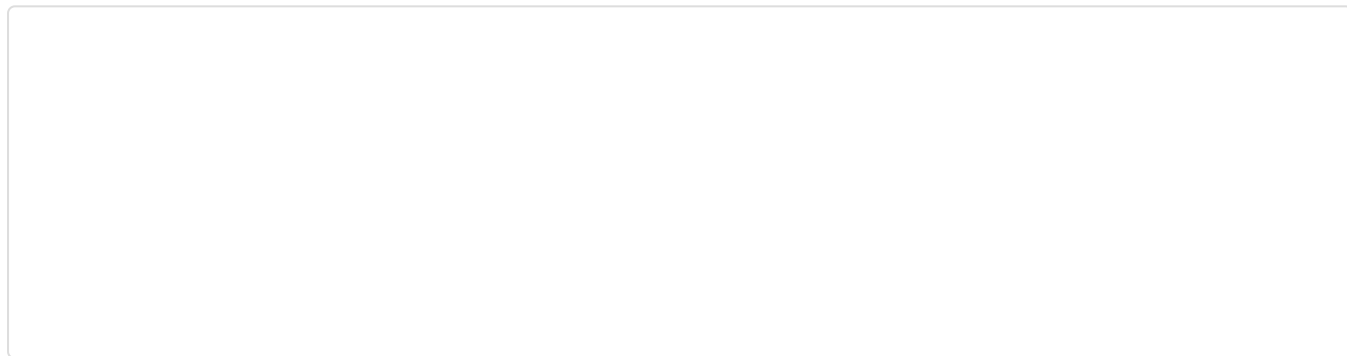
- [] 잘못된 로그인은 401이며 입력 화면에 오류가 표시된다.
- [] 관리자는 제목과 본문을 입력하고 저장할 수 있다.
- [] 제목·본문 제한을 API와 화면에서 검증한다.
- [] 저장 성공 시 상세로 이동하고 공개 목록에 표시된다.
- [] 미인증 쓰기는 401, 인증된 비관리자의 쓰기는 403이다.
- [] 만료된 토큰은 재로그인 안내로 이어진다.
- [] 저장 실패 시 입력을 보존하고 재시도할 수 있다.
- [] H2 통합 테스트와 화면 테스트로 위 동작을 확인한다.

공개 계정 생성 기능은 없지만, 권한 검사 테스트에서는 관리자 권한이 없는 인증 사용자를 구성해 403을 확인할 수 있습니다.

사용 전 → 후



각 티켓에는 파일·완료됐다고 판단할 조건·직접 실행해 확인하는 방법이 있으며, 화살표는 먼저 끝내야 할 티켓의 완료가 필요함을 나타냅니다.



완료 확인: 모든 명세 조건이 티켓에 연결되어 있는지 확인합니다. 서로 상대 작업이 끝나기만 기다리는 순서가 되어서는 안 되고, 배포 티켓에는 필요한 계정·변수·검증 방법까지 있어야 합니다.

6강. /implement — 모든 티켓 구현하고 배포 결과 확인하기

시작 상태: 명세와 티켓 파일이 있습니다. **할 일:** 아래 요청을 같은 대화에 보내 구현과 검사를 맡깁니다. **끝나면:** 실행할 코드와 테스트가 생깁니다. 그 뒤 로컬 화면을 확인하고 배포 준비로 넘어갑니다. 6강은 오래 걸릴 수 있으며 계정 설정이 필요하면 그 지점에서 안내를 따릅니다.

실제 변화 보기 · 6강 변경 내용 · 006: 명세와 티켓이 있던 저장소에 백엔드·프론트·테스트·실행 스크립트·CI가 추가됩니다. [006의 변경 코드](#)와 [게시판 화면](#)을 확인해 보세요.

006 는 구현과 검사를 마친 결과입니다. 구현 중 수행하는 테스트·리뷰·수정은 이 단계에 포함됩니다.

이 실습에서는 티켓 분할을 확인할 때 **모든 티켓을 순서대로 구현한다**는 범위까지 정한 뒤 `/implement` 를 호출합니다. 특정 티켓만 진행하기로 했다면 그 대상을 이어받습니다. 대상이 여러 개여서 불분명할 때만 티켓을 지정합니다.

`/implement-spec` 은 여러 에이전트가 각자의 작업 폴더(worktree)에서 나눠 만든 뒤, 변경을 한 브랜치로 모아 검토 요청(PR)을 만드는 방식입니다. 이 실습은 `/implement` 로 진행합니다.

조립 실습 비유: `/implement` 는 작업 카드를 보고 직접 만들고 시험하는 단계입니다. TDD는 먼저 ‘이 행동이 되어야 한다’는 시험을 만들고, 아직 실패하는지 확인한 뒤, 통과하도록 구현하는 방식입니다. 리뷰는 결과를 다시 살펴 규칙과 약속을 지켰는지 확인하는 과정입니다. 같은 티켓에서 구현 스킬이 이미 테스트와 리뷰를 수행했다면 별도 호출로 반복할 필요는 없습니다.

입력

`/implement`

이 호출은 앞서 정한 구현 범위와 명세·티켓의 완료 조건을 이어받습니다. 기술 구성과 테스트 방식을 다시 입력하지 않습니다. 아래 항목은 구현 결과를 확인하는 기준입니다.

- 합의한 백엔드·프론트 구성과 환경별 DB가 적용되고, 사용 버전과 실행 방법이 기록됩니다.
- HTTP 통합 테스트·화면 테스트·실제 백엔드+H2 E2E의 결과가 남습니다.
- 배포가 포함된 티켓은 실제 배포 결과까지 확인합니다.
- 외부 계정이나 설정값이 부족하면 필요한 항목과 남은 배포 검증을 알려줍니다.
- 실제 검증한 티켓만 완료로 기록합니다. 푸시와 최초 운영 배포는 앞서 정한 진행 범위와 설정 상태를 확인합니다.

구현 결과 확인

에이전트가 안내한 실행 방법으로 앱을 열고, 티켓에 적힌 행동과 기대 결과를 비교합니다. 코드 파일이 생긴 것만으로 티켓이 완료된 것은 아닙니다.

결과물: 화면 변화

직접 확인하고 다음 작업으로 이어가기

공지 작성 티켓이라면 로그인 후 공지를 작성하고, 공개 목록과 상세에 나타나는지 확인합니다. 실패 사례도 티켓의 완료 조건과 대조합니다.

배포가 범위에 포함되어 있다면 실제 배포 결과까지 확인합니다. 계정이나 외부 설정 때문에 막혔을 때는 `/ask-matt` 로 다음 단계를 찾고, 사람이 직접 해야 하는 준비가 있으면 `/wizard` 의 안내를 받습니다. 구체적인 활용은 아래 추가 워크플로우에서 설명합니다.

실행하지 못한 테스트나 배포는 미완료로 남깁니다. 스킬 호출이 끝났다는 사실과 작업의 완료 조건을 만족했다는 사실을 구분합니다.

티켓 완료 표기 예시

```
- **Status:** ready-for-agent
+ **Status:** done
- - [ ] 공지 등록 후 공개 목록에서 확인된다.
+ - [x] 공지 등록 후 공개 목록에서 확인된다.
+
+ ## Comments
+ 실제 테스트 명령·결과와 해당 변경 커밋을 기록한다.
```

done 은 이 튜토리얼에서 추가한 완료 표기입니다. 기본 triage 라벨에 자동으로 포함되는 값이 아닙니다. 외부 설정이 없어 배포하지 못했다면 05를 완료로 바꾸지 않습니다.

한눈에 보는 스킬 사용 결과의 차이

단계	입력	결과물	아직 없는 것
설치	설치 명령	사용할 수 있는 스킬	프로젝트 규칙
/setup-matt-pocock-skills	저장소와 선호	작업 저장 규칙	제품 요구사항 합의
/grill-with-docs	게시판 아이디어·사용할 기술과 지켜야 할 조건	용어·구체적으로 정한 요구사항·결정 기록	통합된 명세
/to-spec	합의된 대화와 문서	검증 가능한 spec.md	구현 단위
/to-tickets	명세	작업의 앞뒤 순서·완료됐다고 판단할 조건이 있는 티켓	실행되는 기능
/implement	전체 티켓	코드·테스트·CI/CD·실행 근거	미실행 배포는 별도 미완료

최종 완료 체크

- ☐ 인터뷰에서 만들 기능과 제외 범위, 완료 기준을 합의했다.
- ☐ 용어와 중요한 결정 이유가 알맞은 문서에 남았다.
- ☐ 명세가 앞선 합의와 일치한다.
- ☐ 티켓에 선후 관계와 확인 가능한 완료 조건이 있다.
- ☐ 구현 결과를 티켓의 조건에 따라 확인했다.
- ☐ 실제 검증 결과와 남은 작업을 기록했다.

추가 사항: /ask-matt로 다음 단계 추천받기

어떤 스킬을 써야 할지 기억나지 않으면 현재 작업 중인 대화에서 물어보세요. `/ask-matt` 는 상황에 맞는 스킬과 진행 경로를 추천하는 안내 역할입니다. 추천을 확인한 뒤 해당 스킬을 호출해 진행합니다.

```
/ask-matt 나 이제 뭐해야 해?
```

예시 1: setup을 마쳤지만 요구사항이 아직 모호할 때

사용자: `/ask-matt` 나 이제 뭐해야 해?

예상 추천: 프로젝트 설정은 끝났습니다. 관리자 권한과 로그인 유지, 삭제 정책을 먼저 정리하도록 `/grill-with-docs`를 추천합니다.

추천받은 스킬을 바로 실행합니다.

```
/grill-with-docs
```

예시 2: 인터뷰가 끝났을 때

```
/ask-matt 나 이제 뭐해야 해?
```

예상 추천은 `/to-spec` 입니다. 이어서 입력합니다.

```
/to-spec
```

예시 3: 티켓까지 만들었을 때

```
/ask-matt 나 이제 뭐해야 해?
```

먼저 해야 하는 작업이 없는 첫 티켓의 `/implement` 를 추천받았다면 다음처럼 진행합니다.

```
/implement
```

첫 티켓 완료 후 다시 `/ask-matt` 나 이제 뭐해야 해? 라고 물으면 남은 티켓과 작업의 앞뒤 순서를 바탕으로 다음 단계를 추천받을 수 있습니다. 이 튜토리얼 6강처럼 모든 티켓을 한 번에 지정할 수도 있고, 추천에 따라 하나씩 진행할 수도 있습니다.

추천은 정해진 답이 아닙니다. 요구가 모호하면 인터뷰, 명세가 준비되면 티켓 분해, 구현 중 오류가 생기면 진단을 추천할 수 있습니다. 새 대화에서는 명세·티켓 경로와 완료한 작업을 함께 알려주면 다음 단계를 판단하기 쉽습니다.

자주 사용되는 워크플로우 10개

여기는 1강부터 이어서 전부 실행하는 추가 수업이 아닙니다. 지금 상황에 맞는 예시 하나만 고르세요. 기존 기능을 고치는 예시는 작동하는 게시판 코드가 있어야 하고, 초기 계획 예시는 코드 없이도 시작할 수 있습니다.

워크플로우는 상황에 맞춰 스킬을 이어 쓰는 순서입니다. 열 가지를 모두 해야 하는 숙제가 아닙니다. 지금 겪는 상황과 가장 비슷한 하나를 골라 시작하세요.

각 항목의 **실무 사례 3개**는 스킬을 고르는 기준을 설명하기 위한 가상 상황입니다. 실제 고객 사례나 이 저장소에서 확인한 구현 결과가 아닙니다. 9번은 질문에 답하는 것만으로 정하기 어려워 조사와 시제품 실험을 여러 번 거쳐야 하는 기획을 다룹니다. 대화와 기록으로 충분하면 `/grill-with-docs` 를 사용합니다.

학교생활 비유: 체육복을 잃어버렸을 때는 ‘마지막으로 본 곳 확인 → 분실물함 확인’을 하고, 축제를 준비할 때는 ‘회의 → 준비물 정리 → 역할 분담’을 합니다. 둘 다 순서가 있지만 목적이 다르듯, 개발도 항상 같은 긴 절차가 필요한 것은 아닙니다.

잠깐: 컨텍스트와 용량은 무엇인가요?

****컨텍스트(context)****는 LLM이 지금 답하거나 작업할 때 참고하는 정보입니다. 현재 대화, 작업 지침, 읽어온 코드와 문서, 도구 실행 결과 등이 포함됩니다. 저장소의 모든 파일이 자동으로 전부 들어오는 것은 아닙니다. 필요한 파일을 읽어야 그 내용이 현재 작업에 전달됩니다.

책상 비유: 프로젝트 폴더는 책상이고, 컨텍스트는 지금 책상 위에 펼쳐놓은 자료입니다. 책상에 책이 있다고 모든 페이지를 동시에 보고 있는 것은 아닙니다. `CONTEXT.md` 는 책상에 있는 ‘우리 팀 용어 사전’이라는 파일 하나이며, 컨텍스트 전체와 같은 뜻이 아닙니다.

****컨텍스트 창(context window)****은 한 번의 처리에서 다룰 수 있는 정보량의 범위입니다. 보통 ****토큰(token)****이라는 단위로 표시합니다. 토큰은 글을 모델이 처리하도록 나눈 조각으로, 한 글자나 한 단어와 항상 같지 않습니다. 한국어·영어·코드에 따라 나뉘는 방식도 달라집니다.

정확한 최대 용량과 실제 사용 가능한 공간은 모델·실행 환경·설정에 따라 달라집니다. ‘항상 몇 글자까지’ 또는 ‘모든 모델은 같은 용량’이라고 외우지 마세요. 사용하는 도구가 제공하는 컨텍스트 사용량·한도·자동 요약 안내를 확인합니다. 구독의 사용량 제한과 한 대화의 컨텍스트 용량도 서로 다른 개념입니다.

책상 크기 비유: 큰 책상에도 책과 공책을 계속 펼치면 새 자료를 놓기 어려워집니다. 긴 로그와 관련 없는 코드까지 쌓으면 중요한 합의를 찾기도 어려워집니다. 그렇다고 책상을 매번 싹 비우면 방금 정한 내용까지 잃습니다. 필요한 자료를 남기고, 작업이 한 단계 끝났을 때 정리하는 것이 좋습니다.

/clear와 /compact는 무엇인가요?

/clear와 /compact는 Matt 스킬이 아니라 사용하는 도구의 대화 관리 기능을 가리킵니다. 지원 여부와 동작은 도구마다 다릅니다.

- /clear는 대화 맥락을 새로 시작하는 기능입니다. 프로젝트 파일을 삭제한다는 뜻은 아닙니다.
- /compact는 필요한 맥락을 요약해 같은 작업을 이어가는 기능입니다. 요약에서 빠질 수 있는 정확한 결정은 원본 문서로 확인합니다.

각 스킬을 호출할 때마다 대화를 초기화할 필요는 없습니다. 이 튜토리얼은 같은 프로젝트와 대화에서 이어갑니다.

워크플로우를 실행하는 방법

공통 준비: 스킬이 설치된 에이전트에서 대상 프로젝트 폴더를 엽니다. “기존 공지”, “수정”, “오류”가 나오는 예시는 이미 만든 게시판이 있어야 합니다. 요청 안의 [실제 경로] 같은 표시는 그대로 입력하지 않고 본인의 파일 경로로 바꿉니다. 경로를 모르면 먼저 에이전트에게 찾아 달라고 하세요.

아래의 A → B는 **A의 결과를 확인한 다음 B를 호출**한다는 뜻입니다. 화살표 전체를 터미널에 붙여 넣지 않습니다. 각 / 스킬은 에이전트 대화창에서 따로 실행합니다.

- 프로젝트의 첫 엔지니어링 작업 전에는 /setup-matt-pocock-skills를 실행합니다. 이미 설정한 저장소에서는 아래 경로의 setup을 반복하지 않습니다.
- 1강 명령으로 Matt Pocock 저장소의 전체 스킬을 Codex·Claude Code에 전역 설치합니다. 아래에는 1강의 소개 표에 없는 스킬도 등장하지만 별도로 골라 설치할 필요는 없습니다. 나중에 추가된 스킬이 필요하다면 같은 전체 설치 명령을 다시 실행합니다.
- 인터뷰 → 명세 → 티켓 → 구현은 같은 대화에서 이어갑니다.
- /implement는 테스트와 리뷰를 포함하는 흐름입니다. 표에서 리뷰를 따로 쓰지 않았다고 검증을 생략하는 뜻은 아닙니다.
- 아래 대화는 추천 예시입니다. 실제 /ask-matt는 대화와 파일의 현재 상태를 보고 다른 경로를 제안할 수 있습니다.

번호	이런 상황이면 선택	핵심 결과
1	기획문서를 받아 개발을 시작해야 한다	문서와 코드의 차이·합의된 명세·티켓·검증된 구현
2	기존 앱의 작은 동작 하나를 바꾸고 싶다	필요한 만큼만 수정한 기능
3	화면이나 기술 조합을 작은 실험으로 확인하고 싶다	서브에이전트의 실험 결과와 선택 근거
4	어떤 기술·서비스가 가능한지 모르겠다	근거 있는 조사와 구현 계획
5	다른 사람이 요청한 일이 쌓였다	정리된 요청과 처리할 작업
6	잘되던 기능이 고장 났다	재현·원인·수정·재발 방지 테스트
7	코드를 바꿀 때마다 여러 곳이 깨진다	구조 개선 계획과 검증된 변경
8	실제 사용자의 답이 없어서 결정을 못 한다	질문지·답변·합의된 명세
9	설계를 정하려면 더 알아보고 여러 실험을 해봐야 한다	조사·시제품·사용자 피드백을 반영한 설계와 명세
10	완성 직전인데 사람만 할 수 있는 설정이 남았다	설정 안내와 실제 배포 확인

1. 추상적인 기획이나 의도로부터 시작할 때

상황: 월요일 오전, 기획자가 ‘환불 관리 개편 v3’ 문서와 화면 시안을 전달하며 이번 주 개발을 요청합니다. 화면에는 환불 버튼과 승인 상태가 있지만 부분 환불, 이미 배송된 주문, 승인자 부재 시 처리는 없습니다. 기존 코드는 전체 취소만 지원합니다. 문서를 곧바로 구현하면 빠진 정책을 개발자나 AI가 임의로 정하게 됩니다.

순서: 기획문서·현재 코드 읽기 → `/grill-with-docs` 로 빠진 조건과 충돌 합의 → `/to-spec` → 명세 검토 → `/to-tickets` → `/implement`.

프로젝트 설정이 없다면 먼저 `/setup-matt-pocock-skills` 로 준비합니다. 기획문서는 에이전트가 실제로 읽을 수 있는 파일이나 접근 가능한 링크로 제공합니다. 읽을 수 없는 첨부·링크를 확인한 것처럼 진행하지 않습니다.

실무 사례 3개:

- **환불 관리 개편 기획서:** 버튼 설명은 ‘선택한 상품 환불’인데 현재 API는 주문 전체를 취소합니다. 상품별 환불 금액, 쿠폰·배송비 배분, 승인 권한, 실패 후 재시도를 질문합니다. 기획 담당자와 합의한 다음 API 변경과 운영 화면의 완성 조건을 명세로 정리합니다. 외부 결제사 지원 여부는 조사할 사실로 분리합니다.
- **영업팀의 견적 승인 화면 기획서:** 화면에는 작성·승인·반려만 있고 ‘승인 후 금액 수정’ 설명이 없습니다. 현재 코드는 승인된 견적도 수정할 수 있습니다. 승인 효력을 유지할지 재승인이 필요한지, 반려 사유를 필수로 할지, 다른 팀 견적을 볼 수 있는지 확인합니다. 기존 동작과 문서가 충돌하는 항목부터 합의합니다.
- **종이 점검표를 앱으로 바꾸는 기획서:** 필수 사진 3장과 제출 버튼은 있는데 현장 통신이 끊겼을 때 설명이 없습니다. 임시 저장, 사진 업로드 실패, 같은 점검의 중복 제출, 제출 후 수정 가능 여부를 질문합니다. 오프라인 동기화까지 필요하면 작은 실험으로 검토합니다. 동기화 방식 조사, 충돌 처리 실험, 현장 사용성 비교까지 거쳐야 설계를 정할 수 있다면 Wayfinder로 이어갑니다.

바로 입력하기: 아래 문서 경로는 실제 제공한 경로로 바꿉니다.

/grill-with-docs

기획문서 docs/planning/refund-v3.md를 기준으로 환불 관리 개편을 시작하자.
먼저 문서와 관련 코드·기존 용어집·ADR을 읽어줘.
문서에 명시된 요구사항, 현재 구현과 충돌하는 내용, 빠진 결정 사항을 구분해줘.
질문마다 기획문서의 해당 절이나 화면 이름을 붙이고,
코드나 공식 자료에서 확인할 사실은 조사하고 내가 결정할 정책을 질문해줘.
내가 답할 수 없는 운영 정책은 담당자에게 확인할 질문으로 남겨줘.

합의 후 같은 대화에서 /to-spec 을 실행합니다. 명세에 원본 문서의 경로·버전과 변경된 합의를 연결하고, 테스트에서 확인할 동작을 검토한 뒤 /to-tickets → /implement 로 이어갑니다. 원본 기획서와 다른 결정을 내렸다면 차이와 이유를 남깁니다.

문서가 이미 충분히 구체적이라면? 문서를 읽고 현재 코드와 대조했을 때 미결정·충돌이 없다면 전체 인터뷰를 반복할 필요는 없습니다. 명세로 정리하거나, 이미 구현용 명세·티켓인 경우 해당 자료를 지정해 구현합니다. /to-spec 은 빠진 정책을 대신 결정하는 스킴이 아닙니다.

끝나면: 기획서의 각 요구사항이 합의된 명세·구현 티켓·검증 결과로 이어지고, 미정인 내용을 구현 완료로 처리하지 않습니다. 문서 없이 처음 만드는 앱은 위 본문의 1-6강처럼 아이디어부터 인터뷰하면 됩니다.

2. 공지 제목이 너무 길 때 오류를 보여주기

상황: 게시판은 이미 있습니다. 제목을 100자보다 길게 쓰면 친절な 안내를 보여주고 싶습니다. 한 번의 대화에서 끝낼 수 있을 만큼 범위가 작습니다.

순서: /grill-with-docs → 합의한 동작과 검증 조건을 짧은 명세로 정리 → /implement

실무 사례 3개:

- **거래처명 길이 제한:** 외부 연동은 80자까지만 받는데 입력 화면은 제한이 없습니다. 공백 제거와 글자 수 계산 기준, 80자 허용·81자 거부, 실패 시 입력 보존을 합의합니다. 짧은 명세를 구현 입력으로 남기면 됩니다.
- **주문 목록 기본 정렬 변경:** 담당자가 최신 주문을 찾느라 매번 정렬합니다. 생성 시각 내림차순과 같은 시각의 보조 정렬 기준, 페이지 이동 시 순서를 확인합니다. 결제·배송 정책까지 인터뷰할 필요는 없습니다.
- **이미 권한이 있는 담당자에게 다운로드 버튼 표시:** 서버의 CSV 권한 검사는 있는데 화면이 역할 하나를 빠뜨렸습니다. 실제 권한과 조회 범위를 확인하고 표시 조건을 수정합니다. 반대로 역할별 데이터 범위를 새로 설계해야 한다면 작은 UI 변경으로 취급하지 않습니다.

/grill-with-docs

공지 제목을 앞뒤 공백 제거 후 1-100자로 제한하고 싶어.
길이 계산, 화면 안내 위치, API 오류 응답을 짧게 확인해줘.
기존 규칙과 충돌하는지도 확인해줘.

인터뷰에서 100자 허용·101자와 빈 제목 거부·실패 시 본문 보존까지 확인한 뒤:

/implement

왜 쓰나요? 작은 변경까지 무조건 큰 명세와 여러 티켓으로 나누면 준비가 작업보다 커질 수 있습니다. 범위가 확실하고 한 대화에서 구현·검증할 수 있으면 짧은 경로가 적합합니다. 예상보다 복잡해지면 명세·티켓 경로로 옮깁니다.

학교생활 비유: 안내문의 낱자 한 줄을 고치는 일에 축제 전체 회의를 다시 열 필요는 없습니다. 다만 낱자가 맞는지와 다른 안내문도 함께 바뀌어야 하는지는 확인해야 합니다.

끝나면: 제한을 넘긴 입력은 저장되지 않고 화면에 이유가 보입니다. 테스트가 100자와 101자처럼 허용되는 길이의 바로 안쪽과 바깥쪽을 확인합니다.

3. 서버에이전트에게 실험을 맡겨 화면이나 기술 조합 확인하기

실무 사례 3개:

- **물류센터 피킹 화면:** 작업자가 장갑을 끼고 품목을 확인합니다. 큰 버튼 목록형과 한 품목씩 넘기는 화면을 `/prototype` 으로 비교해 오조작·긴 상품명·수량 변경을 직접 봅니다. 선택한 화면을 실제 재고 API에 연결하는 일은 이후 구현입니다.
- **부분 취소 상태 모델:** 주문 3개 중 1개 취소 뒤 배송 시작을 눌렀을 때 상태가 이해되는지 실험합니다. 메모리 기반 모델로 사람이 동작을 확인하며, 결제사의 실제 부분 취소 지원 여부는 별도 조사입니다.
- **사내 프록시를 통과하는 파일 업로드:** 20MB 샘플 파일이 테스트 환경의 브라우저 → 프록시 → API로 전달되는지 최소 연결 실험을 맡깁니다. 실제 응답과 크기 제한을 기록합니다. 업로드 화면 모형만 만들고 연동 성공이라고 결론 내리지 않습니다.

상황: 화면을 직접 비교하고 싶거나, 생각한 로직·기술 조합이 가능한지 작은 실험으로 확인하고 싶습니다.

순서: 서버에이전트·별도 worktree 요청 → 화면·로직은 `/prototype`, 기술 연결은 별도 최소 실험 → 결과 확인·선택 → 기록·서버 종료·worktree 정리

전체 요구사항 인터뷰나 명세·티켓 작성부터 시작할 필요는 없습니다. 이번 실험으로 답을 얻고 싶은 질문을 정해 맡기면 됩니다.

서버에이전트 실행은 별도 요청입니다. `/prototype` 만 호출한다고 자동으로 서버에이전트가 만들어지는 것은 아닙니다. 서버에이전트를 지원하는 도구에서 아래처럼 요청합니다.

화면을 비교하고 싶을 때

서버에이전트가 별도 worktree에서 `/prototype`으로 공지 목록의 표형·카드형을 비교할 화면을 만들게 해줘. 휴대폰에서 긴 제목과 낱자가 얼마나 잘 읽히는지 직접 보고 고를게.

에이전트가 안내한 화면을 열어 같은 공지를 두 형태로 비교하고, 선택과 이유를 알려줍니다.

기술 조합이 가능한지 확인하고 싶을 때

서버에이전트에게 React 화면과 Spring API를 연결하는 최소 실험을 맡겨줘. 로그인 후 인증이 필요한 요청을 보내고 결과가 화면에 표시되는지 확인하고 싶어. 실제로 확인한 결과와 아직 확인하지 못한 범위를 알려줘.

/prototype 의 기본 용도는 UI 비교와 로직·상태 모델 실험입니다. **화면 모형만으로 기술 호환성이나 실제 연결 성공을 확인할 수는 없습니다.** 기술 조합을 검증하려면 두 번째 예시처럼 실제로 연결할 대상과 확인할 동작을 지정합니다. 문서상 지원 여부만 궁금하다면 /research 가 더 적합합니다.

끝나면: 비교할 시제품이나 최소 실행 결과, 그 실험이 답한 질문과 한계가 남습니다. 결과를 보고 채택하거나 버립니다. 실제 제품에 반영하기로 결정했을 때 다음 작업을 정하면 됩니다.

화면을 봤다면 무엇이라고 답하나요?

같은 대화에서 주 에이전트에게 선택이나 수정 의견을 말합니다.

카드형이 더 읽기 편해. 이 선택과 이유를 기록해줘.

카드형의 날짜를 오른쪽에 정렬해서 다시 보여줘.

수정이 필요하면 주 에이전트가 후속 작업을 맡깁니다. 서브에이전트는 화면을 만든 뒤 작업을 마쳤을 수 있지만, 사용자의 비교와 선택은 계속 이어집니다. 선택만으로 실제 제품에 반영되는 것은 아닙니다. 제품 반영을 원하면 그 범위를 정해 구현으로 이어갑니다.

실험 종료에는 뒷정리까지 포함합니다

서브에이전트의 작업 종료는 서버 종료나 파일 삭제를 뜻하지 않습니다. 서버는 별도 프로세스이며, 소스코드·worktree·브랜치도 남을 수 있습니다. 서버가 우연히 종료되는 환경도 있으므로 자동 정리를 전제로 하지 않습니다.

대상	실험을 마칠 때 처리
선택과 실험 결과	선택 이유·확인한 동작·미확인 범위를 기록
소스코드	실험 브랜치에 커밋해 다시 참고할 수 있게 보관
미리보기·개발 서버	이번 실험에서 시작한 프로세스와 자식 프로세스를 확인해 종료하고 해당 포트 해제 확인
브라우저	더 볼 필요가 없는 실험 탭 닫기
worktree	미커밋·미추적 파일과 필요한 결과를 보존했는지 확인한 뒤 제거
실험 브랜치	원본 보관용으로 남김. worktree 제거와 브랜치 삭제는 별개

사용자는 길게 정리 절차를 다시 적지 않고 종료 의사를 알려주면 됩니다.

실험 끝. 결과는 보관하고 뒷정리해줘.

주 에이전트는 위 순서로 정리하고 **종료한 서버, 제거한 worktree, 남겨둔 브랜치와 결과 기록**을 보고합니다. 단순히 “완료” 라고만 보고하지 않습니다. 코드까지 폐기하고 싶다면 별도로 명확히 말합니다. 다른 작업의 서버나 파일까지 함께 정리하지 않습니다.

이 매뉴얼에서 실제로 확인한 실험은 **서브에이전트가 worktree 생성 → prototype 스킴 적용 → 표·카드 화면 제작 → 서버 실행 → 주 에이전트의 브라우저 검증**까지입니다. 사용자 선택과 제품 반영, 서버 종료·worktree 제거는 별도 후속 단계입니다. 기술 조합 검증은 이 UI 실험으로 확인한 것이 아닙니다.

4. 배포 서비스에서 무엇이 가능한지 조사한 뒤 결정하기

실무 사례 3개:

- **기업 고객의 SSO 도입 요청:** 해당 인증 공급자의 프로토콜, 필요한 계약 플랜, 테스트 계정 조건을 공식 문서로 조사합니다. 확인된 제약을 받은 뒤 조직별 로그인 정책과 기존 계정 연결 방식을 인터뷰합니다.
- **정산 CSV를 외부 저장소에 보관:** 파일 크기·보관 기간·다운로드 접근 제한이 요구사항입니다. 후보 서비스의 지원 기능과 요금 조건을 출처·확인 날짜와 함께 조사한 뒤 보관 방식을 정합니다. 조사 전 특정 서비스가 조건을 만족한다고 단정하지 않습니다.
- **예약 알림 발송 서비스 선정:** 예약 변경·취소 시 발송 취소가 가능한지, 실패 결과를 어떻게 받는지 조사합니다. 그 결과를 바탕으로 재시도와 사용자 안내 정책을 결정합니다. 조사 문서만으로 실제 발송 성공이 검증되는 것은 아닙니다.

상황: “Railway와 Pages를 같이 써도 로그인과 자동배포가 될까?” 아직 서비스 제약을 잘 모릅니다.

순서: `/research` → `/grill-with-docs` → `/to-spec` → `/to-tickets` → `/implement`

`/research`

공지사항 게시판을 Railway API와 Cloudflare Pages로 나눠 배포하려고 해.
GitHub Actions 배포 방법, 환경변수 주입 시점, 서로 다른 도메인의
인증 연결 제약을 공식 문서로 조사해줘.
확인 날짜와 출처 링크를 포함한 Markdown 파일로 남겨줘.

조사 결과를 확인한 다음:

`/grill-with-docs`

방금 작성한 조사 문서를 기준으로 배포와 인증 방식을 결정하자.
확인된 서비스 제약과 내가 선택해야 하는 조건을 구분해줘.

왜 쓰나요? 모르고 있는 사실은 조사하고, 선택해야 할 것은 함께 결정해야 합니다. ‘공식적으로 가능한가?’와 ‘우리 프로젝트에서 이 방식을 쓸까?’는 서로 다른 질문입니다.

수학여행 비유: 버스 출발 시간과 입장 가능 인원은 찾아볼 사실입니다. 박물관과 과학관 중 어디에 갈지는 조사 후 반에서 정할 선택입니다.

끝나면: 출처가 있는 조사 문서와 그 결과를 반영한 결정·명세가 남습니다. 조사가 끝났다고 앱까지 만들어진 것은 아닙니다.

5. 친구들이 남긴 요청을 정리하고 처리하기

실무 사례 3개:

- **고객센터의 ‘로그인 안 됨’ 접수 12건:** 기기·발생 시각·오류 응답이 없는 건은 정보 보완 대상으로, 재현된 특정 브라우저 오류는 버그로 분류합니다. 12건을 모두 같은 원인의 구현 티켓으로 만들지 않습니다.
- **영업팀의 엑셀 내보내기 요청:** 필요한 열·조회 범위·권한이 빠져 있으면 결정할 내용을 보완합니다. 명확해진 요청에 실행 가능한 설명을 붙이고 준비된 작업부터 구현합니다.
- **외부 기여자의 버그 수정 PR:** 프로젝트가 외부 PR도 접수 대상으로 다루는 경우, 실제 재현 문제와 수정 범위·검증 근거를 확인해 후속 작업을 정리합니다. triage 완료를 병합 완료로 표현하지 않습니다.

상황: “휴대폰에서 글자가 잘려요”, “검색이 있으면 좋겠어요”, “로그인이 안 돼요” 같은 요청이 한꺼번에 들어왔습니다.

순서: /triage → 필요한 정보 보완 → 준비된 티켓 /implement

```
/triage
다른 사용자가 등록한 게시판 요청들을 분류해줘.
정보가 부족한 요청과 바로 구현 가능한 요청을 구분하고,
현재 프로젝트의 라벨 규칙을 사용해줘.
```

왜 쓰나요? ‘로그인이 안 돼요’만으로는 비밀번호 오류인지 서버 장애인지 알 수 없습니다. 먼저 정보를 보완해야 하는 요청과 바로 실행할 수 있는 요청을 나눕니다.

보건실 비유: 들어온 학생을 모두 같은 순서로 치료하기 전에 무슨 일이 있었는지 듣고 필요한 처치를 구분합니다. 요청을 분류하는 단계이지 모든 문제를 그 자리에서 구현하는 단계는 아닙니다.

구현 준비가 끝난 실제 티켓 경로를 지정해 실행합니다.

```
/implement
```

끝나면: 요청마다 현재 상태와 필요한 후속 행동이 보이고, 준비된 요청부터 구현합니다. /to-tickets 가 만든 티켓은 이미 실행 가능한 형태이므로 다시 triage할 필요가 없습니다.

6. 공지 수정 버튼을 눌렀는데 500 오류가 날 때

실무 사례 3개:

- **주소가 없는 주문만 상세 조회 500:** 실패 주문과 정상 주문의 데이터·요청을 비교하고 해당 조건을 재현합니다. 로그와 코드로 원인을 확인한 뒤 같은 조건이 통과하는지 검증합니다. 처음부터 null 처리 문제라고 확정하지 않습니다.
- **배포 후 특정 API만 응답 지연:** 동일 요청의 변경 전후 시간과 쿼리·외부 호출 증거를 확보합니다. DB 문제인지 외부 API 지연인지 좁힌 뒤 수정합니다. 성능 증거 없이 전체 구조를 재설계하지 않습니다.
- **결제 알림 재전송 때 주문 중복 생성:** 같은 이벤트를 두 번 전달하는 재현 절차를 만들고 중복이 생기는 지점을 확인합니다. 수정 후 재전송·동시 도착에서도 기대한 주문 수가 유지되는지 검사합니다.

상황: 어제는 됐는데 오늘은 수정 저장이 실패합니다. 화면만 보고는 왜 그런지 모릅니다.

순서: /diagnosing-bugs → 오류 재현 → 원인 확인 → 수정·같은 오류가 다시 생기지 않는지 확인하는 테스트

여기서 '재현 → 원인 확인 → 수정'은 `/diagnosing-bugs` 안에서 진행하는 단계입니다. 각각이 별도의 스킬 이름은 아닙니다.

`/diagnosing-bugs`

관리자로 로그인해서 기존 공지 제목을 수정하고 저장하면 500이 나.

같은 문제를 다시 확인할 수 있는 테스트나 명령부터 확보해줘.

실제 요청과 오류 로그를 근거로 원인을 좁히고 수정해줘.

확인된 사실과 아직 추측인 원인을 구분해줘.

왜 쓰나요? 원인을 모른 채 여러 곳을 고치면 우연히 잠깐 해결되거나 다른 기능이 깨질 수 있습니다. 같은 실패를 반복 확인할 방법이 있어야 수정 전과 후를 비교할 수 있습니다.

자전거 비유: 자전거가 덜컹거린다고 바퀴·브레이크·안장을 모두 바꾸지 않습니다. 어느 상황에서 소리가 나는지 재현하고 느슨한 부분을 찾은 뒤 고칩니다. 같은 오류가 다시 생기지 않는지 확인하는 테스트는 나중에 같은 나사가 다시 풀리지 않았는지 확인하는 점검표입니다.

끝나면: 같은 재현 절차가 수정 전에는 실패하고 수정 후에는 통과합니다. 단순한 새 기능 요청처럼 인터뷰부터 전부 다시 시작할 필요는 없습니다.

7. 작은 수정인데 다섯 군데를 고쳐야 할 때

실무 사례 3개:

- **배송비 규칙이 장바구니·주문·견적에 중복:** 무료 배송 기준을 바꿀 때 세 화면의 금액이 달라집니다. 실제 변경 이력과 호출 코드를 보고 책임을 모을 후보를 고릅니다. 화면별 규칙이 정말 같은지부터 확인합니다.
- **파일 저장소 교체에 호출부 20곳 수정:** 업로드 URL 생성·저장·삭제 세부 처리가 곳곳에 노출돼 있습니다. 어떤 세부사항을 모듈 내부로 숨길지 검토하고, 저장·조화·삭제 동작을 보장할 테스트 위치를 합의합니다.
- **할인 테스트가 내부 함수 이름에 종속:** 함수를 나눌 때마다 테스트가 깨지지만 최종 가격 오류는 놓칩니다. 공개된 계산 동작으로 검증할 수 있는 구조 개선 후보를 찾습니다. 단지 파일 수를 줄이는 리팩터링을 목표로 삼지 않습니다.

상황: 제목 제한을 바꿀 때마다 화면·API·여러 서비스에서 같은 숫자를 고칩니다. 한 군데를 빠뜨리면 오류가 생깁니다.

순서: `/improve-codebase-architecture` 안에서 후보 조사·선택·grilling → 필요하면 `/grill-with-docs` 로 남은 논의와 용어·결정 기록 보완 → `/to-spec` → `/to-tickets` → `/implement`

원문 스킬은 후보를 보여주는 데서 끝나지 않고, 선택한 후보를 grilling 으로 논의합니다. 이미 합의한 내용을 다음 인터뷰에서 처음부터 반복할 필요는 없습니다.

`/improve-codebase-architecture`

공지 제목 검증을 바꿀 때 여러 곳을 수정해야 해.

현재 코드에서 책임이 흩어졌거나 중복된 부분을 찾아

구조 개선 후보와 이유를 보여줘.

후보를 확인하고 고른 다음:

/grill-with-docs

방금 제안한 공지 검증 구조 개선을 구체화하자.

사용자가 보는 동작은 유지하고 내부 구조를 정리하고 싶어.

어떤 책임을 모으고 무엇을 테스트로 보장할지 합의해줘.

왜 쓰나요? 고장이 날 때마다 복사된 코드를 하나씩 고치는 대신, 변경이 어려운 구조 자체를 살펴볼 수 있습니다. 개선 후보를 찾는 단계와 실제 구조를 바꾸는 단계를 구분합니다.

교실 정리 비유: 가위를 찾을 때마다 다섯 서랍을 뒤지는 상황이라면 매번 빨리 찾는 연습만 할 것이 아니라 공용 가위를 둘 장소와 관리 규칙을 정하는 편이 낫습니다.

끝나면: 같은 사용자 동작이 유지된다는 검증과 함께 변경 지점이 명확해집니다. 단순히 파일 수가 줄었다고 무조건 좋은 구조는 아닙니다.

8. 선생님이 답해줘야 요구사항을 정할 수 있을 때

실무 사례 3개:

- **고객센터의 환불 승인 정책:** 금액별 승인자와 이미 배송된 주문의 예외를 운영 책임자가 정해야 합니다. 실제 주문 상황을 넣은 질문지를 만들고 받은 답을 토대로 요구사항을 합의합니다.
- **물류센터의 바코드 재출력:** 파손 라벨 재출력 때 기존 라벨을 무효화하는지 현장 담당자에게 확인해야 합니다. 개발자가 추측하지 않고 중복 스캔 상황을 질문지에 넣습니다.
- **기업 고객의 관리자 인수인계:** 유일한 관리자가 퇴사하면 누가 권한을 넘겨받을지 고객 운영 책임자의 답이 필요합니다. 질문지를 전달하고 답변을 받아 미응답·충돌 항목을 추가 확인한 뒤 명세로 옮깁니다.

상황: 공지를 누가 올리고, 누가 삭제할지 개발자인 내가 결정할 수 없습니다. 실제 사용하는 선생님의 답이 필요합니다.

이 스킬은 질문지 Markdown 파일을 만드는 것까지만 합니다. 외부인에게 자동으로 전송하거나 답변을 받아오지 않습니다.

순서: /to-questionnaire → 내가 질문지를 전달하거나 회의에서 질문 → 내가 답변을 수집해 에이전트에 제공 →

/grill-with-docs → /to-spec → /to-tickets → /implement

담당	할 일
에이전트	질문 대상과 목적을 확인하고 to-questionnaire-<주제>.md 파일 작성
나	질문지를 메일·메신저 등으로 직접 전달하거나 회의에서 사용
답변할 사람	질문에 답변
나	받은 답변을 대화에 붙여 넣거나 답변 파일을 에이전트에 제공
에이전트	제공받은 답변을 바탕으로 요구사항 정리

혼자 요구사항을 결정할 수 있다면 이 단계를 생략하고 /grill-with-docs 로 진행하면 됩니다. 고객이나 담당자에게 확인할 질문을 준비해야 할 때 사용하는 보조 스킬입니다.

/to-questionnaire

게시판을 사용할 선생님께 질문지를 드리려고 해.
누가 공지를 작성·삭제하는지, 삭제 복구가 필요한지,
공지 수정 내역을 남겨야 하는지 결정에 필요한 질문을 정리해줘.
먼저 질문 대상과 질문지의 목적을 나와 확인해줘.

받은 답변을 에이전트에 제공한 후:

/grill-with-docs

선생님이 보내준 답변을 기준으로 게시판 요구사항을 정리하자.
답변끼리 충돌하거나 아직 모호한 부분을 확인해줘.

왜 쓰나요? 실제 운영자가 결정할 일을 AI나 개발자가 추측해서 정하면 나중에 다시 만들게 됩니다. 질문지를 작성하는 것과 답을 얻는 것은 별도 단계입니다.

학교생활 비유: 반 티셔츠를 주문할 때 내 생각으로 친구들의 사이즈를 정하지 않습니다. 필요한 항목을 설문으로 받고 빠진 응답을 확인한 뒤 주문합니다.

끝나면: 실제 사용자의 답을 근거로 명세를 작성할 수 있습니다. 답변이 도착하지 않았다면 그 결정은 아직 미완료입니다.

9. 그릴위드독스보다 더 알아보고 실험해야 할 때 — /wayfinder

Wayfinder는 질문에 답하는 것만으로 설계를 정하기 어려워, 조사와 시제품 실험을 여러 번 거쳐야 하는 큰 기획에 사용합니다. 알아낸 사실과 직접 써본 결과를 반영해 다음에 무엇을 확인하고 결정할지 이어갑니다.

예를 들어 “이 화면이 편할까?”는 직접 눌러봐야 답할 수 있고, “이 저장 방식이 가능할까?”는 기존 코드와 기술 제약을 확인해야 답할 수 있습니다. 이 과정이 기획의 상당 부분을 차지하면 Wayfinder로 조사·실험·논의를 함께 진행합니다.

지금 필요한 것	사용할 스킬	진행 방식
질문에 답하며 원하는 기능과 조건 정리	/grill-with-docs	인터뷰하며 용어와 설계 결정을 기록
어떤 실험을 할지 이미 정했고, 시제품 하나가 필요함	/prototype	만들어서 써보고 판단
여러 조사와 실험의 결과를 봐야 설계 방향을 정할 수 있음	/wayfinder	조사·시제품·추가 논의를 연결하고, 결과에 따라 기획을 보완

/grill-with-docs 를 쓰면서 필요할 때 직접 /research 나 /prototype 으로 이어가도 됩니다. **Wayfinder**는 이런 조사와 실험이 여러 갈래로 필요한 기획의 진행까지 맡기고 싶을 때 선택합니다. 기획한 날짜 수나 질문 개수만으로 선택하지 않습니다.

영상에서 확인할 수 있는 사용 방식

영상에서 Matt는 자신의 앱에 명령 팔레트를 추가하며 아이콘 선택, 다른 다이어그램 검색, 요소 재사용 등을 다룹니다. 기능을 정하기 위해 필요한 조사와 논의를 나눠 진행합니다. [실제 사례 5:40부터](#)

Wayfinder는 필요한 경우 `research`, `prototype`, `grilling` 을 사용합니다. 시제품을 직접 사용한 피드백도 설계에 반영합니다. [스킬별 역할 7:41부터](#)

설계가 정리되면 `/to-spec` 으로 결과를 모읍니다. 영상에서는 Wayfinder가 개발 흐름에서 `/grill-with-docs` 와 같은 자리에 들어간다고 설명합니다. [다음 단계 10:25부터](#)

내부에서 사용하는 기능

무엇이 필요한가요?	Wayfinder의 진행	사용자가 할 일
모르는 사실이나 외부 기술 제약 확인	필요할 때 <code>research</code> 로 조사	조사 결과를 바탕으로 선택할 조건 확인
화면이나 동작을 직접 보고 판단	필요할 때 <code>prototype</code> 으로 시제품 제작	직접 써보고 의견 주기
목적·규칙·설계 선택 구체화	<code>grilling</code> + <code>domain-modeling</code> 으로 논의·기록	질문에 답하고 결정하기
실험에 필요한 계정·자료·환경 준비	가능한 작업을 수행하고 사람만 할 일 안내	필요한 자료나 접근 환경 준비

이 과정에서 사용자가 매번 `/research` 나 `/prototype` 을 따로 골라 입력할 필요는 없습니다. 다만 시제품에 대한 평가와 기획 결정에는 사용자의 참여가 필요합니다. 첫 실행에서 목표와 필요한 조사·실험을 정리한 뒤, 후속 작업도 `/wayfinder` 로 이어가며 결과를 확인합니다.

실무 사례 1 — 다이어그램 앱의 명령 팔레트

가상 상황: `Ctrl+K` 로 아이콘, 문서, 저장한 다이어그램 요소를 찾고 싶습니다. 검색 결과를 섞을지 나눌지 확신이 없고, 저장한 요소를 다른 문서에서 재사용할 수 있는지도 확인해야 합니다.

조사·실험·결정: 검색 데이터 출처 확인 → 여러 검색 화면을 시제품으로 만들어 사용 → 선택한 요소의 저장·재사용 방식 조사와 실험 → 결과를 보고 검색 흐름과 복사 동작 확정.

시제품 하나만 비교하면 끝나는 경우에는 `/prototype` 으로 충분합니다. 화면 실험과 데이터 조사 결과를 서로 반영해야 전체 기능을 정할 수 있을 때 Wayfinder로 진행합니다. 이 상세 시나리오는 영상의 명령 팔레트 사례를 바탕으로 구성한 활용 예시입니다.

실무 사례 2 — 강의 영상에서 짧은 영상 만들기

가상 상황: 강의 영상을 넣으면 유용한 구간을 찾아 잘라주는 개인 도구를 만들고 싶습니다. 자동 추천의 효과와 사람이 후보를 검토하는 방식 모두 아직 확신이 없습니다.

조사·실험·결정: 음성 인식과 영상 처리 제약 조사 → 실제 샘플로 구간 추천 실험 → 후보를 고르는 화면 시제품 사용 → 문장 중간이 잘리지 않는지 확인 → 결과에 따라 자동 추천과 직접 선택의 범위 결정.

대본만으로 추천한 구간이 이해되지 않는다면 화면 분석을 조사할지, 사람이 직접 고르는 방식으로 좁힐지 다시 논의합니다. 실험 결과가 다음에 알아볼 내용을 바꾸는 기획입니다.

실무 사례 3 — 통신이 끊겨도 쓰는 현장 점검 앱

가상 상황: 사진을 찍고 점검표를 제출하는 앱을 만들고 싶지만, 현장의 통신이 불안정합니다. 저장·재전송·중복 제출을 어떻게 처리해야 할지 실제 사용 조건을 확인해야 합니다.

조사·실험·결정: 기기 저장과 업로드 제약 조사 → 오프라인 입력·재연결 실험 → 같은 점검을 두 번 수정하는 상황 재현 → 현장에서 써보기 → 저장 시점·충돌 처리·제출 완료 표시 확정.

단순히 “오프라인 지원”이라고 명세에 적기 전에, 조사와 실험에서 확인한 동작을 바탕으로 범위를 정합니다.

입력과 다음 단계

```
/wayfinder
다이어그램 편집기에 명령 팔레트를 추가하고 싶어.
아이콘 검색, 다른 다이어그램 검색, 저장한 요소 재사용이 필요해.
화면 구성과 저장 방식은 아직 확신이 없어.
필요한 조사와 시제품 실험을 진행하고,
내가 직접 써본 피드백을 반영해서 설계를 구체화해줘.
목표는 첫 버전의 구현 명세를 작성할 수 있을 만큼 결정하는 것이야.
```

진행: /wayfinder 로 필요한 조사·시제품·논의를 수행하고 결과를 확인 → 설계가 정리되면 /to-spec → /to-tickets → /implement.

Wayfinder 다음에 /grill-with-docs 를 반드시 다시 실행할 필요는 없습니다. 내부에서 이미 필요한 인터뷰를 진행합니다. 끝나면 조사 근거, 시제품 피드백, 합의한 설계가 남고, 그 결과로 구현 명세를 작성할 수 있습니다. 정식 기능 구현과 제품 검증은 이후 단계에서 진행합니다.

10. 구현은 끝났는데 계정·배포 설정에서 막혔을 때

실무 사례 3개:

- **운영 도메인의 DNS 권한 부재:** 코드와 배포 환경은 준비됐지만 도메인 관리자는 다른 팀입니다. 사람이 입력할 레코드와 확인 절차를 안내하고, 변경 뒤 접근 가능한 DNS·HTTPS 결과를 검증합니다.
- **SSO 관리자 동의 필요:** 테스트 연동은 됐지만 고객 조직 관리자가 앱을 승인해야 합니다. 해당 담당자만 할 수 있는 절차를 안내하고, 승인 후 실제 로그인·권한 흐름을 다시 확인합니다.
- **운영 결제 계정 승인 대기:** 샌드박스 테스트가 통과해도 운영 계정 승인이 남아 있습니다. 사람이 처리할 계정 절차와 자격증명 설정 위치를 안내합니다. 안내 스크립트 생성이나 샌드박스 성공을 운영 결제 검증 완료라고 쓰지 않습니다.

상황: 로컬 앱과 테스트는 되지만 Railway 계정 연결이나 Cloudflare 토큰 발급처럼 사람이 직접 처리해야 하는 단계가 남았습니다.

순서: `/implement` 중 누락 설정 확인 → `/wizard` → 설정 완료 → `/implement` 재개·배포 검증

```
/ask-matt 나 이제 뭐해야 해?  
앱 구현과 로컬 테스트는 끝났고 배포 티켓만 남았어.  
외부 서비스 계정 승인과 배포 토큰 설정이 필요하대.  
에이전트가 할 수 있는 작업과 내가 직접 해야 하는 작업을 구분해줘.
```

`/wizard` 를 추천받았고 실제로 사람이 해야 하는 일이 남았다면:

```
/wizard  
배포 티켓에서 내가 직접 해야 하는 계정 승인과 토큰 설정만  
한 단계씩 따라갈 수 있는 안내로 만들어줘.  
각 단계의 목적과 성공 확인 방법을 설명하고,  
비밀값은 채팅에 붙여 넣지 않고 필요한 저장 위치에 설정하게 해줘.  
내 운영체제에 맞는 실행 방법도 안내해줘.
```

왜 쓰나요? 계정 승인이나 실제 계정·토큰 입력은 코드 작성과 다른 종류의 작업입니다. 설정을 ‘알아서 됐겠지’라고 넘기지 않고 필요한 값과 위치를 정확히 확인합니다. 에이전트가 직접 할 수 있는 일에는 이 우회 단계를 추가하지 않습니다.

학교 방송 비유: 방송 원고와 장비 점검이 끝나도 방송실 열쇠를 가진 선생님의 승인이 필요할 수 있습니다. 열쇠를 받는 절차와 방송을 실제로 내보내는 절차를 구분하는 것입니다.

`/wizard` 는 설치된 스킬 기준으로 Bash 안내 스크립트를 생성하는 도구입니다. macOS에서는 터미널에서 Bash로 실행하고, Windows에서는 Git Bash·WSL 등 필요한 실행 환경을 확인합니다. PowerShell에서 그대로 실행된다고 가정하지 않습니다. 설정을 끝내면 남은 티켓을 지정해 `/implement` 를 재개하고 실제 배포·공개 화면·DB 저장 결과를 확인합니다.

끝나면: 설정값이 필요한 곳에 들어가고 배포가 확인됩니다. 안내 파일만 생성됐다면 아직 배포 완료가 아닙니다.

열 가지 중에서도 못 고르겠다면

아래 네 줄이면 시작할 수 있습니다. 대괄호 부분을 지금 상황으로 바꾸세요.

```
/ask-matt 나 이제 뭐해야 해?  
만들고 싶은 것: [예: 학교 축제 공지 게시판]  
끝낸 것: [예: setup과 요구사항 인터뷰]  
막힌 것: [예: 명세를 만들지, 화면부터 실험할지 모르겠어]  
쉬운 비유로 추천 이유를 설명하고 다음에 복사할 명령을 알려줘.
```

추천을 받으면 **왜 이 단계인지 읽고** → **추천 스킬을 실행하고** → **결과물을 확인**합니다. 어려운 말 때문에 멈췄다면 `/wait-what` 으로 방금 설명을 쉽게 다시 풀어달라고 요청할 수도 있습니다.

```
/wait-what  
방금 말한 '테스트에서 입력과 결과를 확인할 위치'와 '먼저 끝내야 할 티켓'을  
공지사항 게시판의 실제 예와 학교생활 비유로 다시 설명해줘.
```

이 매뉴얼을 실제로 실행해 본 결과

강좌 번호와 같은 커밋 번호

2강은 002, 3강은 003처럼 강좌 번호와 태그 번호가 같습니다. 커밋 링크를 열면 해당 강에서 추가하거나 바꾼 파일이 바로 보입니다. 초록색은 추가, 빨간색은 삭제입니다.

쉬운 비유: 교과서의 강 번호와 실습 결과물의 이름표를 같게 붙였습니다. 4강을 읽고 004 를 열면 그 강에서 만든 명세가 보입니다.

태그	수업	결과	해당 강의 변경 내용
002	2강 · 하네스 세팅	작업 규칙과 에이전트 안내	002 커밋
003	3강 · 요구사항 정리	CONTEXT.md와 ADR	003 커밋
004	4강 · 명세	기능·완성 조건을 담은 spec.md	004 커밋
005	5강 · 티켓	작업 5개와 실행 순서	005 커밋
006	6강 · 구현·검사·배포	게시판 코드·테스트·CI·배포 설정과 공개 결과	006 커밋

설치는 PC 환경에 적용되므로 별도 코드 커밋이 없습니다. 실제 앱의 실행·배포 방법과 검증 기록은 별도 저장소에서 확인합니다.

[실행 프로젝트 안내](#) · [운영 상태와 증거](#)